

從函式物件、型別提示到裝飾器與策略模式

這份手冊把四大主題串成一條完整的學習路線：先確立「函式是一級物件」這個地基，接著替函式貼上型別提示的標籤，再深入閉包與裝飾器的運作原理，最後用策略模式示範「函式取代類別」的實戰重構。每段程式碼都附上逐行說明與好懂的比方，建議照順序讀，前面的觀念會在後面不斷被接起來用。

第一篇 把函式當成物件

函式是一級物件

先講個好懂的比方。在 Python 裡，函式跟整數、字串、字典一樣，都是「正港的物件」，這種身分學術上叫「一級物件」。你可以把它想成手上一張隨身的工具卡：能現場辦一張出來（執行時建立）、塞進皮夾裡（指派給變數）、當禮物遞給別人（當引數傳進另一個函式），也能當回禮被退回來（從函式回傳）。函式能享有這四種待遇，就是這個課程要你看懂的重點，而這也是函數式語言（像 Clojure、Elixir、Haskell）的看家本領，後來連 JavaScript、Go、Java 8 都把它學了過去。

順帶一提，這個課程比舊版多談了兩種「可呼叫物件」，從七種變成九種，多出來的是原生協程跟非同步產生器；另外也補上了 Python 3.8 才有的「僅限位置參數」，還把型別提示的執行期讀取挪到後面講。這些先有個印象就好。

接著看程式。第一段是要證明「函式本身就是物件」：

```
def factorial(n):  
    """returns n!"""  
    return 1 if n < 2 else n * factorial(n - 1)  
  
factorial(42)  
factorial.__doc__  
type(factorial)
```

這串三引號文字會存進 __doc__，等於函式的說明書
n<2 直接回傳 1，否則 n 乘上「(n-1) 的階乘」——函式呼叫自己，這叫遞迴
算 42!，Python 整數沒有上限，會吐出一長串幾十位的大數字
撈出上面那句 'returns n!'，help() 顯示的說明就是讀這個
<class 'function'>，白紙黑字證明 factorial 是 function 類別生出來的一個物件

這邊有兩個點容易卡住。第一個是 `1 if n < 2 else ...`，這是條件運算式，整行會先算出一個值再回傳，不是 if 區塊，所以可以直接接在 `return` 後面。第二個是它「自己叫自己」的遞迴：要算 5! 得先算 4!，要算 4! 又得先算 3!，一路退到 $n < 2$ 才開始往回乘，這個收尾條件要是少了，就會無限叫下去停不下來。

再來看「一級」這個身分實際怎麼用：

```
fact = factorial          # 注意沒有括號！這是把「函式本身」貼上另一個名牌，不是呼叫它
fact(5)                  # 用新名字一樣叫得動，得到 120
list(map(factorial, range(11))) # 把 factorial 當引數丟進 map，對 0~10 每個數各算一次階乘
# [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800]
```

最關鍵的差別就在那個括號：`fact = factorial` 是把函式整個搬過去，`fact()` 才是真的去執行它。能這樣搬來搬去、還能當引數傳給 `map`，就是「函式是一級物件」最直接的證據。另外 `map(函式, 可迭代物件)` 回傳的是一個 `map` 物件，它很懶，不會馬上算，要用 `list()` 包起來逼它把每一項真的跑出來。

如果想寫得更有 Python 味，同樣的邏輯、同樣對 0~10 取階乘，社群更習慣用串列推導式取代 `map`：

```
[factorial(n) for n in range(11)] # 結果跟上面那行 list(map(...)) 一模一樣，但讀起來更直覺
```

`map` 是從函數式語言流過來的味道，推導式則是 Python 自家更順口的講法；兩種都對，挑你讀起來最清楚的那個用就行。

高階函式

先給個比方。所謂「高階函式」，就是那種會「吃別的函式」或「吐出別的函式」的函式——它把函式當食材或當成品在用。最好懂的就是排序：你遞給 `sorted` 一把「量尺」（也就是 `key` 那個函式），它就照你給的尺去量每一項，再決定誰排前誰排後。尺換了，順序就跟著換，但東西本身一根寒毛都不會動到。這個 `key` 的用法，前面課程在講 `list.sort` 跟 `sorted` 差別時就帶過了。

先看照「長度」排：

```
fruits = ['strawberry', 'fig', 'apple', 'cherry', 'raspberry', 'banana']
sorted(fruits, key=len) # key=len 把 len 當量尺，照「字串長度」由短到長排；len 沒加括號，是把函式本身交出去
```

```
# ['fig', 'apple', 'cherry', 'banana', 'raspberry', 'strawberry']
```

這邊的關鍵是：`key` 收的是一個「單引數函式」，`sorted` 會把每個元素丟進去，拿回傳值當排序依據，而不是拿原本的值來比。

再換一把尺，照「倒著拼的字串」排：

```
def reverse(word):  
    return word[::-1]      # [::-1] 是切片，步長 -1，等於把字串整個倒過來  
  
reverse('testing')        # 'gnitset'  
sorted(fruits, key=reverse) # 量尺換成「倒拼字串」，所以尾巴拼法像的會湊在一起，做押韻字典就是這招  
# ['banana', 'apple', 'fig', 'raspberry', 'strawberry', 'cherry']
```

要特別注意：清單裡的字一個都沒被改，只是「拿倒拼當比較基準」而已，所以那幾種莓果才會排在一塊。換句話說，任何單引數函式都能拿來當 `key`。

函數式世界裡最有名的高階函式是 `map`、`filter`、`reduce`、`apply`。其中 `apply` 在 Python 3 已經拿掉了，因為現在要用一組動態引數呼叫函式，直接寫 `fn(*args, **kwargs)` 就好，不需要它。剩下三個還在，但多數情況都有更順口的寫法，往下看就懂：

```
list(map(factorial, range(6)))          # 對 0~5 每個數各取階乘  
# [1, 1, 2, 6, 24, 120]  
  
[factorial(n) for n in range(6)]        # 同一件事，改成串列推導式，讀起來更直覺  
# [1, 1, 2, 6, 24, 120]  
  
list(map(factorial, filter(lambda n: n % 2, range(6)))) # 先用 filter 篩出奇數(n%2 餘 1 視為真)，再對它們取階乘  
# [1, 6, 120]  
  
[factorial(n) for n in range(6) if n % 2] # 推導式一行就把 map 跟 filter 兩件事做完，連 lambda 都省掉  
# [1, 6, 120]
```

這裡兩個觀念要抓住。一個是 `n % 2`：奇數取餘數是 1 當成真、偶數是 0 當成假，所以 `filter` 留下來的就是奇數。另一個是 Python 3 的 `map` 跟 `filter` 其實回傳的是產生器（一種迭代器），所以最貼切的替代品是產生器運算式，要真的拿到串列才需要再包 `list()`。一條推導式就能把篩選加轉換一次做完，還不用另外寫 `lambda`，這就是現在大家比較推薦的講法。

至於 `reduce`，在 Python 3 被從內建降級到 `functools` 模組裡，它最常見的用途就是加總，可是加總改用內建的 `sum` 又快又好讀：

```
from functools import reduce    # Python 3 把 reduce 搬進 functools 模組
from operator import add        # add 就是「把兩個相加」這個動作的函式版本

reduce(add, range(100))        # 把 0~99 兩兩相加、一路累積滾成一個值
# 4950

sum(range(100))                # 同樣是加總，但 sum 是內建，效能跟可讀性都贏
# 4950
```

`reduce` 的精神就是「拿一個雙引數函式，把整串資料一路滾成單一結果」，加總只是最常見的那種。同一掛把序列收斂成一個值的內建還有 `all` 跟 `any`：`all` 是全部都為真才回 `True`（空的也回 `True`），`any` 是只要有一個為真就回 `True`（空的回 `False`）。`reduce` 更完整的用法，後面課程在做 Vector 的雜湊跟加速比較時，會擺進有意義的脈絡裡再講一次。

有時候為了餵給高階函式，你只需要一個用過就丟的小函式，這正是「匿名函式（`lambda`）」存在的理由，下個課程就會講到它。

匿名函式

先講個比方。`lambda` 做出來的是「匿名函式」，你可以把它想成現場找來的免洗小幫手：不用正式 `def` 取個名字、不用另外掛牌，當場捏一個出來、用完就丟。你只是要把每個字倒拼一下，犯不著為這點小事正式請一位有名有姓的廚師，現場剝個洋蔥就走人。

但這個免洗小幫手有個硬規矩：`lambda` 的身體裡只能放「純運算式」，也就是一條會算出值的式子。像 `while`、`try` 這種陳述句都不能塞，連用 `=` 做指派都不行，因為指派本身也算陳述句。Python 3.8 之後的指派運算式 `:=`（外號海象運算子）倒是能用，可是話說回來，要是你的 `lambda` 已經複雜到得動用它，那就代表它太難讀了，該乖乖改寫回有名字的 `def` 函式。

`lambda` 真正的好用時機，就是當高階函式的引數。把前面那個「倒拼排序」拿來改寫，連 `reverse` 都不用先定義：

```
fruits = ['strawberry', 'fig', 'apple', 'cherry', 'raspberry', 'banana']
sorted(fruits, key=lambda word: word[::-1]) # 直接在 key 現場捏一個沒名字的函式，把每個字倒拼當量尺
# ['banana', 'apple', 'fig', 'raspberry', 'strawberry', 'cherry']
```

效果跟前一個課程那段「先 `def` 一個 `reverse` 再傳進去」一模一樣，只是這裡省去了取名字的工。`lambda word: word[::-1]` 拆開來看就是：收一個引數 `word`，回傳 `word[::-1]`（倒拼的字串），冒號前面是參數、後面是要回傳的那條式子，整段就這麼簡單。

要記住的是：除了「當高階函式的引數」這種場合，`lambda` 在 Python 其實很少派得上用場。語法限制太死，稍微複雜一點的 `lambda` 不是難讀就是根本寫不出來。所以只要你的 `lambda` 開始讓人皺眉頭，就照 Fredrik Lundh 的重構建議來處理：先寫一句註解講清楚它在幹嘛，再從那句註解想一個能抓住精神的好名字，接著把它改寫成用那個名字的 `def` 函式，最後把註解刪掉。一條 `lambda` 能被你順順地命名，往往就表示它本來就該是個正式函式。

還有一個更乾淨的進階做法跟這同樣邏輯：如果你的 `lambda` 只是「去拿某個欄位或某個屬性」，例如 `key=lambda x: x[0]` 或 `key=lambda x: x.name`，可以改用 `operator` 模組裡的 `itemgetter(0)`、`attrgetter('name')`，效果一樣，但讀起來更清楚、跑起來也更快。只是像上面這種「切片倒拼」沒有對應的現成工具，留著 `lambda` 反而最直接。

其實 `lambda` 只是 Python 裡眾多「可呼叫物件」的其中一種而已，下個課程就會把它們全部整理過一遍。

九種「可呼叫物件」

先給個好懂的比方。Python 裡那個「呼叫運算子」就是函式後面那對括號 `()`，你可以把它想成一顆「啟動鈕」——按下去，東西就會動起來做事。問題是，能按這顆鈕的不是只有函式，Python 裡有好幾種東西後面都能接 `()`。想知道某個物件到底「按不按得下去」，用內建的 `callable()` 問一下就知道：

```
[callable(obj) for obj in (abs, str, 13)] # abs 是內建函式、str 是類別，都能呼叫；13 是整數，按不動
# [True, True, False]
```

到 Python 3.9 為止，資料模型文件總共列出九種「可呼叫物件」，意思就是這九種東西後面都接得上那顆啟動鈕。

最直覺的前四種是這樣：第一種是「使用者自訂函式」，也就是你用 `def` 或 `lambda` 親手做出來的；第二種是「內建函式」，這種是用 C 實作給 Python 用的，像 `len`、`time.strftime`；第三種是「內建方法」，一樣是 C 寫的方法，像 `dict.get`；第四種就是一般「方法」，也

就是定義在類別主體裡頭的那些函式。

接著兩種跟類別有關，特別要留意。第五種是「類別」本身：當你呼叫一個類別，它會先跑 `__new__` 生出一個實例，再跑 `__init__` 把它初始化，最後把這個實例交還給你；因為 Python 根本沒有 `new` 這個運算子，所以「呼叫類別」用起來就跟「呼叫函式」一個樣。第六種是「類別的實例」：只要那個類別有定義 `__call__` 方法，它生出來的實例就能像函式一樣被叫，而這正好就是下個課程的主題。

最後三種比較特別，是產生器跟非同步那一掛。第七種是「產生器函式」，主體裡有用到 `yield` 的函式或方法，呼叫它不會直接給你答案，而是回傳一個產生器物件。第八種是「原生協程函式」，用 `async def` 定義，呼叫時回傳一個協程物件。第九種是「非同步產生器函式」，一樣用 `async def`，但主體裡又有 `yield`，呼叫時會回傳一個非同步產生器，要搭 `async for` 來用，這是 Python 3.6 才加進來的。

這後面三種跟其他可呼叫物件有個本質差別：它們回傳的東西從來都不是「你要的資料本身」，而是「還得再處理一輪才能擠出資料或真正做事」的物件。產生器函式回傳的是迭代器，這部分後面的課程會專門講；原生協程跟非同步產生器回傳的物件，則一定要靠像 `asyncio` 這種非同步框架才動得起來，也會留到後面的課程細談。

把這九種摸過一遍之後，接下來要動手的，就是去打造那種「實例本身就能被當函式呼叫」的物件，也就是第六種背後的 `__call__`。

自訂可呼叫型別：讓物件也能像函式一樣被叫

不只 Python 的函式是真正的物件，反過來，任意一個物件也能被你調教成「像函式一樣會做事」。要做到這件事，你只需要實作一個 `__call__` 方法就夠了。

先給個比方。想像一台賓果搖號機，或是尾牙抽獎箱：裡頭一堆號碼牌洗得亂七八糟，你伸手一抽就拿走一張，抽走一張就少一張，這個「還剩哪些」的狀態會一路被記著。而 `__call__` 的魔法在於——它讓你「直接對箱子按一下啟動鈕」就等於「伸手抽一張」，也就是 `bingo()` 跟 `bingo.pick()` 變成同一件事。

```
import random

class BingoCage:
    def __init__(self, items):
        self._items = list(items)      # 收任何可迭代物件，複製一份成自己的串列；複製是為了不去動到外面傳進來的那個清單
        random.shuffle(self._items)    # 因為 self._items 保證是 list，shuffle 這種就地洗牌才一定能用

    def pick(self):
```

```

try:
    return self._items.pop()      # 主力方法：從尾端取出一張並回傳，取出後箱子裡就少一張
except IndexError:
    raise LookupError('pick from empty BingoCage') # 箱子空了 pop 會丟 IndexError，這裡改丟訊息更清楚的例外

def __call__(self):
    return self.pick()           # 捷徑：讓 bingo() 直接等同 bingo.pick()

```

這段有幾個地方容易讓人愣了一下。第一個是 `list(items)`：傳進來的可能是 `range`、是別人手上的清單、是任何可迭代的東西，這裡刻意複製一份自己的，是怕之後洗牌、抽牌時去動到外面那個原本的清單，留下莫名其妙的副作用。第二個是 `random.shuffle`，它是「就地」把串列順序打亂，而它能保證運作的前提就是你給它一個 `list`，所以前行先轉成 `list` 不是多此一舉。第三個是那個例外接力：`pop` 在空串列上會丟 `IndexError`，這裡接住它、改丟一個帶自訂訊息的 `LookupError`，讓用的人一看就知道是「對空箱子抽牌」。

實際玩一下，重點在於 `bingo` 這個實例真的可以像函式一樣被呼叫，而且內建的 `callable()` 也認得它是可呼叫的：

```

bingo = BingoCage(range(3))    # 用 0、1、2 三張牌建一台機器
bingo.pick()                   # 正常的抽法
bingo()                        # 直接呼叫實例，效果跟 pick() 一樣
callable(bingo)                # True，證明這個實例已經是可呼叫物件

```

像這樣實作 `__call__`，是做出「長得像函式、但又能在多次呼叫之間記著內部狀態」的物件最省事的方式——這裡的內部狀態就是「箱子裡還剩哪些牌」。`__call__` 另一個很好用的場合是寫裝飾器：裝飾器本身一定得是可呼叫的，而且有時候需要在多次呼叫之間「記住」某些東西，例如做記憶化（把昂貴運算的結果快取起來，下次直接拿），或是把一個複雜的實作拆成好幾個方法分工。

順帶提一個同樣邏輯、但走另一條路的進階做法：要做「帶內部狀態的函式」，函數式的對應寫法是用閉包，不必開一個類別。閉包跟裝飾器都會留到後面的課程專門講，到時候你就會發現它跟今天這台賓果機在精神上是同一回事。

把可呼叫物件摸熟之後，接下來要進的是 Python 在「宣告函式參數」跟「把引數傳進去」這件事上提供的那套強大語法。

從位置參數到僅限關鍵字參數

Python 函式最讚的特色之一，就是那套彈性大到誇張的參數處理機制；跟它密切相關的，還有呼叫函式時用 `*` 跟 `**` 把可迭代物件和對映拆開成一個個獨立引數的玩法。

先給個比方。把函式的簽名想成餐廳點餐的分流關卡：第一個一定是「主餐名稱」，這是固定位置；主餐後面你隨便加幾道配菜，全會被一個袋子（`*content`）打包成 tuple 收走；你臨時指定、菜單上沒列的調味（沒在簽名裡具名的關鍵字引數），由另一個袋子（`**attrs`）裝成 dict；而 `class_` 這道比較特別，只能「指名點」，沒辦法用排隊位置硬塞給它。下面這個 `tag` 就是一台萬用 HTML 標籤產生器：

```
def tag(name, *content, class_=None, **attrs):
    """Generate one or more HTML tags"""
    if class_ is not None:                # class 是 Python 保留字，只能用 class_ 接住，再塞回屬性字典
        attrs['class'] = class_
    attr_pairs = (f' {attr}="{value}"'    # 把每個屬性排成 ' key="value"'，sorted 讓輸出順序固定
                  for attr, value in sorted(attrs.items()))
    attr_str = ''.join(attr_pairs)       # 把所有屬性片段接成一整串
    if content:                          # 有內容，就為每一段內容各包一個有頭有尾的標籤
        elements = (f'<{name}{attr_str}>{c}</{name}>' for c in content)
        return '\n'.join(elements)
    else:                                # 沒內容，就回傳一個自閉合的單一標籤
        return f'<{name}{attr_str} />'
```

簽名這一行就是整段的精華：`name` 是普通位置參數；`*content` 把「第一個之後的所有位置引數」全收成一個 tuple；`class_=None` 因為排在 `*content` 後面，自動變成「僅限關鍵字」參數，預設給 `None`；`**attrs` 則把所有沒在簽名裡點名的關鍵字引數收成 dict。靠這四樣搭配，同一個函式就能被千奇百怪地呼叫：

```
tag('br')                # 只給一個位置引數 → 產生該名字的空標籤
# '<br />'

tag('p', 'hello')         # 'hello' 被 *content 收成 ('hello',)
# '<p>hello</p>'

print(tag('p', 'hello', 'world'))  # 第一個之後的全進 *content，有幾段就包幾個標籤
# '<p>hello</p>'
```

```
# <p>world</p>

tag('p', 'hello', id=33)          # 沒列在簽名裡的 id=33，被 **attrs 收成 {'id': 33}
# '<p id="33">hello</p>'

print(tag('p', 'hello', 'world', class_='sidebar')) # class_ 只能用具名方式傳進來
# <p class="sidebar">hello</p>
# <p class="sidebar">world</p>

tag(content='testing', name='img') # 連第一個 name 都能改用具名方式給
# '<img content="testing" />'

my_tag = {'name': 'img', 'title': 'Sunset Boulevard',
          'src': 'sunset.jpg', 'class': 'framed'}
tag(**my_tag)                      # ** 把字典每一項拆成獨立的具名引數餵進去
# ''
```

最後那個 `tag(**my_tag)` 有個小巧思值得注意：字典裡可以大大方方放一個 `'class'` 鍵，因為它純粹是個字串鍵，不會跟保留字 `class` 撞名；拆開後 `name` 會綁到 `name` 參數，剩下的 `title`、`src`、`class` 通通落進 `**attrs`。

僅限關鍵字參數是 Python 3 的特色。要指定它，就把參數寫在某個 `*` 開頭的參數後面。如果你根本不想收可變數量的位置引數、卻又想要僅限關鍵字參數，那就在簽名裡單獨放一個 `*`：

```
def f(a, *, b):                  # 單獨一個 * 表示後面的 b 是「僅限關鍵字」；這裡沒給預設值，代表 b 是必填
    return a, b

f(1, b=2)                       # (1, 2)，b 乖乖用具名方式傳
f(1, 2)                         # TypeError: b 不收位置引數，所以它說「只吃 1 個位置引數，你卻給了 2 個」
```

這裡要記住一點：僅限關鍵字參數不一定要有預設值，它可以是必填的，就像上面這個 `b`。

僅限位置參數

從 Python 3.8 起，自訂函式的簽名也能指定「僅限位置參數」。這個特性其實內建函式老早就有了，像 `divmod(a, b)` 只能用位置方式呼叫，你沒辦法寫成 `divmod(a=10, b=4)`。要做到這件事，就在參數列裡放一個 `/`：

```
def divmod(a, b, /): # / 左邊的 a、b 都是「僅限位置」，不准寫成 divmod(a=10, b=4)
    return (a // b, a % b)
```

規則很單純：`/` 左邊的全是僅限位置，`/` 之後你還能照常擺其他參數。套回剛才那個 `tag`，如果想逼 `name` 只能用位置方式傳，就寫成這樣：

```
def tag(name, /, *content, class_=None, **attrs):
```

更多僅限位置參數的用法，可以去翻「What's New In Python 3.8」跟 PEP 570。把參數這套機制摸熟之後，這部分課程接下來會繼續往函數式風格的應用走。

函數式程式設計的好幫手套件

雖然 Guido 講得很明白，他當初設計 Python 並不是要把它做成函數式語言，但靠著一級函式、模式比對，再加上像 `operator` 這類套件撐腰，要走函數式風格其實也走得有模有樣。

operator 模組

先給個比方。`operator` 模組就像一間五金行：你需要「乘法」這個動作當成工具拿去用時，不必自己折一支鐵絲（手寫 `lambda`），架上早就有現成的 `mul` 等你拿。而 `itemgetter`、`attrgetter`、`methodcaller` 則是店裡附設的「客製小工廠」——你跟它說要第幾欄、要哪個屬性、要叫哪個方法，它當場幫你打一支專用的小工具出來。

函數式寫法裡常常想把算術運算子當函式用。比方說你想把一串數字相乘來算階乘、又不想用遞迴：加總有現成的 `sum`，可是乘法沒有對應的內建函式。你可以動用前面課程講過的 `reduce`，但 `reduce` 需要你餵它一個「把兩項相乘」的函式，於是只好臨時捏一個 `lambda`：

```
from functools import reduce
```

```
def factorial(n):  
    return reduce(lambda a, b: a * b, range(1, n + 1)) # reduce 把 1~n 一路兩兩相乘，lambda 就是那個「相乘」動作
```

而 `operator` 早就幫你把幾十種運算子都做成函式了，這種 `lambda a, b: a*b` 的小工具根本不用自己寫，直接抓 `mul` 來用：

```
from functools import reduce  
from operator import mul  
  
def factorial(n):  
    return reduce(mul, range(1, n + 1)) # mul 就是現成的「相乘」函式，比手寫 lambda 乾淨
```

`operator` 還能取代另一掛「只會一招」的 `lambda`，也就是「從序列挑出某項」或「從物件讀某個屬性」的那種。`itemgetter` 跟 `attrgetter` 就是專門生這種小函式的工廠。下面用 `itemgetter` 把一串 tuple 照「國碼」（第 1 欄）排序：

```
metro_data = [  
    ('Tokyo', 'JP', 36.933, (35.689722, 139.691667)),  
    ('Delhi NCR', 'IN', 21.935, (28.613889, 77.208889)),  
    ('Mexico City', 'MX', 20.142, (19.433333, -99.133333)),  
    ('New York-Newark', 'US', 20.104, (40.808611, -74.020386)),  
    ('São Paulo', 'BR', 19.649, (-23.547778, -46.635833)),  
]  
  
from operator import itemgetter  
for city in sorted(metro_data, key=itemgetter(1)): # itemgetter(1) 等於做一支「拿索引 1 那項」的函式  
    print(city)  
# ('São Paulo', 'BR', 19.649, (-23.547778, -46.635833))  
# ('Delhi NCR', 'IN', 21.935, (28.613889, 77.208889))  
# ('Tokyo', 'JP', 36.933, (35.689722, 139.691667))  
# ('Mexico City', 'MX', 20.142, (19.433333, -99.133333))  
# ('New York-Newark', 'US', 20.104, (40.808611, -74.020386))
```

`itemgetter(1)` 等價於 `lambda fields: fields[1]`，但讀起來、寫起來都更省事。如果你丟給它好幾個索引，它做出來的函式就會回傳一個「把那幾項都抓出來」的 `tuple`，拿來做多重鍵排序很方便：

```
cc_name = itemgetter(1, 0)          # 一次抓索引 1 跟 0，回傳（國碼，城市名）
for city in metro_data:
    print(cc_name(city))
# ('JP', 'Tokyo')
# ('IN', 'Delhi NCR')
# ('MX', 'Mexico City')
# ('US', 'New York-Newark')
# ('BR', 'São Paulo')
```

因為 `itemgetter` 骨子裡用的是 `[]` 運算子，所以它不只吃序列，連對映、還有任何有實作 `__getitem__` 的類別都通吃。

`itemgetter` 有個兄弟叫 `attrgetter`，它做的函式是「照名字去拿物件的屬性」。同樣地，給它好幾個屬性名就回傳一個 `tuple`；更厲害的是，只要名字裡帶了 `.`（點），它還會一路鑽進巢狀物件裡去把屬性挖出來：

```
from collections import namedtuple
LatLon = namedtuple('LatLon', 'lat lon')          # 先定義經緯度
Metropolis = namedtuple('Metropolis', 'name cc pop coord') # 再定義都會

metro_areas = [Metropolis(name, cc, pop, LatLon(lat, lon)) # 注意這裡的巢狀拆解：把（lat, lon）拆出來組成 coord
                for name, cc, pop, (lat, lon) in metro_data]

metro_areas[0].coord.lat    # 35.689722，先示範手動鑽進去拿緯度

from operator import attrgetter
name_lat = attrgetter('name', 'coord.lat')       # 做一支「拿 name 跟巢狀的 coord.lat」的函式
for city in sorted(metro_areas, key=attrgetter('coord.lat')): # 再用一支來照緯度排序
    print(name_lat(city))
# ('São Paulo', -23.547778)
# ('Mexico City', 19.433333)
```

```
# ('Delhi NCR', 28.613889)
# ('Tokyo', 35.689722)
# ('New York-Newark', 40.808611)
```

`operator` 裡大概列了 54 個名字，多數一看就懂。其中一群以 `i` 開頭再接另一個運算子名的（像 `iadd`、`iand`），對應的是擴增指派運算子（也就是 `+=`、`&=` 這些）：如果第一個引數是可變的，它們會就地改動它；如果不可變，那就跟沒有 `i` 前綴的版本一樣，單純把運算結果回傳。

剩下的 `operator` 函式裡，最後要看的是 `methodcaller`。它跟 `attrgetter`、`itemgetter` 同樣是「當場做一支函式」，差別在於它做出來的函式會「照名字去呼叫物件的某個方法」：

```
from operator import methodcaller
s = 'The time has come'

upcase = methodcaller('upper')          # 做一支「呼叫 .upper()」的函式
upcase(s)                               # 'THE TIME HAS COME'

hyphenate = methodcaller('replace', ' ', '-') # 多給的引數會一起綁進去當呼叫參數
hyphenate(s)                             # 'The-time-has-come'
```

第一個其實只是給你看 `methodcaller` 怎麼運作；如果你純粹想把 `str.upper` 當函式用，直接拿類別來呼叫、把字串當引數傳就好：`str.upper(s)` 一樣會給你 `'THE TIME HAS COME'`。第二個那行更值得注意——它先把 `replace` 的兩個引數綁死了，這種「先凍結部分引數」的把戲，正好就跟 `functools.partial` 做的事一樣，也就是接下來要講的東西。

用 `functools.partial` 凍結引數

`functools` 模組提供好幾個高階函式，前面課程看過的 `reduce` 是其一，另一個就是 `partial`：你給它一個可呼叫物件，它會生出一個新的可呼叫物件，並且把原本那些引數中的一部分先綁定成固定值。這在「想把一個要好幾個引數的函式，硬塞進只接受少引數回呼的 API」時特別好用。

接續剛才的比方，`partial` 就像「先把醬料調好的半成品醬包」：原本要好幾樣材料的函式，先幫你固定其中幾樣，之後只要補齊剩下的就能直接用。

```
from operator import mul
from functools import partial

triple = partial(mul, 3)          # 從 mul 生一支新函式，把第一個位置引數先綁成 3
triple(7)                        # 21
list(map(triple, range(1, 10)))  # mul 本身沒辦法直接餵給 map，但綁好的 triple 可以
# [3, 6, 9, 12, 15, 18, 21, 24, 27]
```

更實用的一個情境是 Unicode 正規化。如果你常常要在比較或儲存字串前，先對它套 `unicodedata.normalize('NFC', s)`，那做一支固定好 'NFC' 的 `nfc` 函式就很順手：

```
import unicodedata, functools

nfc = functools.partial(unicodedata.normalize, 'NFC') # 把第一個引數 'NFC' 凍起來
s1 = 'café'
s2 = 'cafe\u0301'      # 這是 e 後面接一個「組合用的尖音符」，看起來一樣，碼點卻不同
s1 == s2                # False，因為兩串的底層碼點不一樣
nfc(s1) == nfc(s2)      # True，正規化之後才會被認定相等
```

`partial` 的用法就是：第一個引數放可呼叫物件，後面跟著任意數量、要綁定的位置引數和關鍵字引數。拿前面課程那個 `tag` 函式來示意，一次凍結一個位置引數加一個關鍵字引數：

```
from tagger import tag
from functools import partial

picture = partial(tag, 'img', class_='pic-frame') # 把第一個位置引數綁成 'img'，class_ 綁成 'pic-frame'
picture(src='wumpus.jpeg')                       # 只要補上 src 就能用
# ''
```

```
picture          # functools.partial(<function tag at 0x...>, 'img', class_='pic-frame')
picture.func      # <function tag at 0x...>, 原本那支函式
picture.args      # ('img',), 被綁住的位置引數
picture.keywords  # {'class_': 'pic-frame'}, 被綁住的關鍵字引數
```

`partial()` 回傳的是一個 `functools.partial` 物件，它身上的 `func`、`args`、`keywords` 這幾個屬性，就讓你能回頭看到原本那支函式跟被凍住的引數各是什麼。`functools` 裡還有一個兄弟 `partialmethod`，做的事跟 `partial` 一樣，只是它是設計來搭配方法用的。

除此之外，`functools` 模組裡還有一些天生就是要拿來「當裝飾器」用的高階函式，後面的課程會帶你看它們怎麼實作。

第二篇 函式裡的型別提示

先給個比方。型別提示就像你在每個抽屜外面貼上標籤，註明「這格放襪子、那格放文件」。貼了標籤，旁邊的工具就能在你還沒拉開抽屜之前，先幫你揪出「欸你這雙襪子塞錯格了」這種錯；可是不貼也照樣能用，因為這標籤從頭到尾都是選配，不是強制。

型別提示是 Python 自從 2001 年發布的 Python 2.2 把型別和類別統合起來之後，歷史上最大的一次改變。不過它並不是對每個 Python 使用者都同等有用，正因為如此，它才應該永遠保持可選。PEP 484 替函式的引數、回傳值跟變數，引進了「明確宣告型別」的語法和語意，目的是讓開發工具能透過靜態分析去揪出程式裡的臭蟲——也就是不必真的把程式跑過測試，光看程式碼就能抓錯。

最大的受益者，是那些用 IDE（整合式開發環境）跟 CI（持續整合）的專業軟體工程師。但讓這群人覺得型別提示划算的那套成本效益算盤，套到所有 Python 使用者身上就不成立了，因為 Python 的使用族群比這寬太多：科學家、交易員、記者、藝術家、自造者、分析師，還有各個領域的學生等等都在用。對他們之中的大多數人來說，學型別提示的成本恐怕更高——除非本來就會某個帶有靜態型別、子型別和泛型的語言；而好處又更低，因為他們跟 Python 互動的方式、加上手上程式碼規模較小、團隊也小（常常是一個人的「團隊」），都讓型別提示的價值打了折。在探索資料和點子的場合，像資料科學、創意運算、還有單純學習的時候，Python 預設的動態型別反而更簡單、更能讓人盡情表達。

這個課程聚焦在「函式簽名」裡的型別提示，至於型別提示搭配類別怎麼用、以及 `typing` 模組其他功能，會留到後面的課程處理。這個課程主要會帶你走過這幾件事：先動手用 `Mypy` 實際玩一遍「漸進式型別」；再看「鴨子型別」跟「名義型別」這兩種互補的觀點；接著綜覽標

註裡可能出現的幾大類型別，這一塊大約佔了整個課程六成的篇幅；然後是替可變參數（`*args`、`**kwargs`）加型別提示；最後談型別提示與靜態型別的限制跟缺點在哪。

關於漸進式型別

延續前面那個貼標籤的比方：漸進式型別的精神，就是你可以這個抽屜先貼標籤、那個抽屜先空著，想到哪貼到哪、一格一格慢慢來，不必一口氣把整櫃都貼滿。

PEP 484 替 Python 帶進了一套漸進式型別系統。其他同樣走漸進式型別路線的語言，還有微軟的 TypeScript、Google 為 Flutter SDK 打造的 Dart，以及 Facebook 用 HHVM 虛擬機支援的 PHP 方言 Hack。至於 Mypy 這個型別檢查器，它一開始其實是一門語言——一種帶有漸進式型別、還有自己直譯器的 Python 方言；後來是 Guido van Rossum 說服了 Mypy 的作者 Jukka Lehtosalo，把它改造成一個專門檢查「已標註型別的 Python 程式」的工具。

一套漸進式型別系統有三個關鍵性格。第一，它是可選的：只要你的程式沒寫型別提示，檢查器預設就不該對它發警告；遇到它判斷不出型別的物件，它會直接假設那是 `Any` 型別，而 `Any` 被視為「跟所有型別都相容」，等於那格還沒貼標籤的抽屜，工具就當它什麼都能放，不囉嗦。第二，它不會在執行期抓型別錯誤：型別提示是給靜態型別檢查器、程式碼檢查工具（linter）和 IDE 用來發警告的，它並不能阻止程式真正跑起來時，把不對的值傳進函式或指派給變數。第三，它不會讓程式跑更快：型別標註提供的資訊，理論上是可以拿來在產生的位元組碼裡做優化，但截至 2021 年 7 月，作者所知的 Python 執行環境都還沒有真的實作這種優化。

漸進式型別在使用上最棒的一點，就是型別標註從頭到尾都是可選的。在靜態型別系統裡，大部分的类型限制很好表達，不少會有點麻煩，有些很難，還有少數根本表達不出來。你很可能寫出一段很棒的 Python 程式，測試覆蓋率漂亮、測試也全過，卻偏偏沒辦法替它加上能讓型別檢查器滿意的型別提示。那也沒關係，把那幾個搞不定的型別提示拿掉、照樣出貨就好。

而且這份「可選」是貫穿每一個層級的：你可以有整個套件完全不寫型別提示；當你把這種套件匯入到一個有在用型別提示的模組時，可以叫檢查器對它閉嘴；你甚至能加上特定的註解，要檢查器忽略你程式裡指定的某幾行。

實戰漸進式型別

延續前面貼標籤的比方：這一段我們找一個簡單的函式，在 Mypy 這位「驗標籤稽核員」的盯場下，一格一格慢慢把標籤貼上去，看漸進式型別在實務上到底怎麼運作。

要動手的對象是 `show_count`，它會回傳一個字串，依數量湊出單數或複數的講法：

```
show_count(99, 'bird')    # '99 birds'
show_count(1, 'bird')     # '1 bird'
show_count(0, 'bird')     # 'no birds'
```

還沒貼任何標籤的原始版本長這樣：

```
def show_count(count, word):
    if count == 1:
        return f'1 {word}'
    count_str = str(count) if count else 'no'
    return f'{count_str} {word}s'

# 剛好 1 個，單數，不加 s
# 0 的時候講 'no'，其餘照數字
# 其他情況一律加 s 變複數
```

從 Mypy 開始

要開始型別檢查，就對 `messages.py` 這個模組下 `mypy` 指令。預設設定下，Mypy 看這份程式不會挑出任何毛病，因為函式簽名上一個標籤都沒有，Mypy 預設就直接跳過它不管——除非你另外設定。

搭配的還有用 `pytest` 寫的單元測試，放在 `messages_test.py`，一樣還沒貼標籤：

```
from pytest import mark
from messages import show_count

@mark.parametrize('qty, expected', [
    (1, '1 part'),
    (2, '2 parts'),
])
def test_show_count(qty, expected):
    got = show_count(qty, 'part')
    assert got == expected

def test_show_count_zero():
```

```
got = show_count(0, 'part')
assert got == 'no parts'
```

把 Mypy 調嚴一點

命令列選項 `--disallow-untyped-defs` 會讓 Mypy 把「任何沒替全部參數和回傳值貼標籤的函式定義」都標成錯。對測試檔下這個選項，會吐出三個錯誤加一條提示：

```
$ mypy --disallow-untyped-defs messages_test.py
messages.py:14: error: Function is missing a type annotation
messages_test.py:10: error: Function is missing a type annotation
messages_test.py:15: error: Function is missing a return type annotation
messages_test.py:15: note: Use "-> None" if function does not return a value
Found 3 errors in 2 files (checked 1 source file)
```

不過剛起步玩漸進式型別時，作者偏好換另一個比較溫和的選項 `--disallow-incomplete-defs`。一開始它什麼話都不說：

```
$ mypy --disallow-incomplete-defs messages_test.py
Success: no issues found in 1 source file
```

這時就能只先替 `show_count` 補上回傳型別：

```
def show_count(count, word) -> str:
```

光是這一筆，就足以讓 Mypy 開始正眼看它。用一樣的指令再檢查一次，Mypy 就會回頭去看 `messages.py`：

```
$ mypy --disallow-incomplete-defs messages_test.py
messages.py:14: error: Function is missing a type annotation for one or more arguments
Found 1 error in 1 file (checked 1 source file)
```

`--disallow-incomplete-defs` 的好處就在這：你可以一個函式一個函式慢慢貼標籤，而那些你還沒去動的函式並不會被它念。一個能讓 Mypy 滿意的完整簽名長這樣：

```
def show_count(count: int, word: str) -> str:
```

一個有預設值的參數

原本的 `show_count` 只對「規則名詞」有效。要是複數沒辦法靠加 `s` 拼出來，就該讓使用者自己提供複數形：

```
show_count(3, 'mouse', 'mice')    # '3 mice'
```

來玩一下「型別驅動開發」。先加一個會用到第三個引數的測試，記得替測試函式補上回傳型別 `-> None`，不然 Mypy 不會檢查它：

```
def test_irregular() -> None:
    got = show_count(2, 'child', 'children')
    assert got == '2 children'
```

Mypy 馬上抓到問題，因為函式現在只收兩個引數：

```
$ mypy messages_test.py
messages_test.py:22: error: Too many arguments for "show_count"
Found 1 error in 1 file (checked 1 source file)
```

接著就改 `show_count`，補上一個有預設值的複數參數：

```
def show_count(count: int, singular: str, plural: str = '') -> str:
    if count == 1:
        return f'1 {singular}'
    count_str = str(count) if count else 'no'
```

```
if not plural:                                # 沒給複數形，就用最常見的「加 s」湊一個
    plural = singular + 's'
return f'{count_str} {plural}'
```

現在 Mypy 就回報「Success」了。順帶提幾個型別提示的好風格：參數名稱跟 `:` 之間不留空白、`:` 後面留一個空白；而在「有型別提示」的參數那個預設值前面的 `=`，兩邊都要留空白。反過來說，PEP 8 規定如果某個參數沒有型別提示，它的 `=` 兩邊就不該留空白。

用 None 當預設值

上面那版裡，`plural` 標成 `str`、預設值是空字串 `''`，型別沒衝突，這寫法作者也喜歡。但換到別の場合，`None` 往往是更好的預設值——尤其當這個選用參數預期收的是「可變型別」時，`None` 幾乎是唯一合理的預設，這點前面課程在講「拿可變型別當參數預設值是個壞主意」時談過。要讓 `plural` 預設為 `None`，簽名會變成這樣：

```
from typing import Optional

def show_count(count: int, singular: str, plural: Optional[str] = None) -> str:
```

拆開來看：`Optional[str]` 表示 `plural` 可以是字串、也可以是 `None`；而你必須明確寫出預設值 `= None`，因為要是你不給 `plural` 預設值，Python 執行期就會把它當成「必填參數」。記住一個關鍵——執行期其實是不理會型別提示的，型別提示管不到「必填還是選填」這件事，那是由「有沒有給預設值」決定的。另外要記得從 `typing` 模組匯入 `Optional`；匯入型別時，習慣用 `from typing import X` 這種寫法，可以讓函式簽名短一點、好讀一點。

對漸進式型別有了第一手的實際感受之後，下一步就要來想想：「型別」這個概念，在實務上到底指的是什麼。

型別是由「支援哪些操作」定義的

關於「型別」這個概念，文獻上有很多種定義。PEP 483 給的說法大意是：一個型別，就是「一組值」加上「一組能套用在這些值上的函式」。但實務上，更好用的想法是——直接把「它支援哪些操作」當成一個型別的決定性特徵。

舉個例子，從「能做哪些操作」的角度看，下面這個函式裡的 `x` 可以是哪些型別？

```
def double(x):  
    return x * 2
```

`x` 可以是各種數值（`int`、`complex`、`Fraction`、`numpy.uint32` 等等），也可以是序列（`str`、`tuple`、`list`、`array`），還可以是 N 維的 `numpy.array`，或任何「有實作或繼承一個能接受 `int` 引數的 `__mul__` 方法」的型別。換句話說，能不能當 `x`，看的是它支不支援「乘法」這個操作，而不是它叫什麼名字。

可是一旦你這樣替它貼標籤（先別管缺的回傳型別，專注看參數）：

```
from collections import abc  
  
def double(x: abc.Sequence):  
    return x * 2
```

型別檢查器反而會把這段判定為錯。你跟 Mypy 說 `x` 是 `abc.Sequence`，它就會把 `x * 2` 標成錯誤，因為 `Sequence` 這個抽象基底類別並沒有實作或繼承 `__mul__`。實際在執行期，這段對具體的序列（`str`、`tuple`、`list`、`array`）和數字都跑得動，因為執行期根本不理會型別提示；但檢查器只認你「明明白白宣告」的東西，而 `abc.Sequence` 身上就是沒有 `__mul__`。這正是這一節標題的由來：型別是由它支援哪些操作來定義的。

在漸進式型別系統裡，會同時牽扯到兩種看待型別的眼光。第一種是「鴨子型別」，這是 Smalltalk 這門物件導向先驅語言、以及 Python、JavaScript、Ruby 採用的觀點：物件有型別，但變數（包含參數）沒型別。實務上，物件登記成什麼型別不重要，重要的是它「實際支援哪些操作」——只要我能呼叫 `birdie.quack()`，那在這個情境裡 `birdie` 就是隻鴨。按定義，鴨子型別只在執行期、真的去對物件動手做操作時才會被把關，所以它比名義型別靈活，代價是讓更多錯誤拖到執行期才爆。

第二種是「名義型別」，這是 C++、Java、C# 採用、也被「有標註的 Python」支援的觀點：物件和變數都有型別。但物件只活在執行期，而型別檢查器只在乎「原始碼裡那些被貼上型別提示的變數和參數」。假設 `Duck` 是 `Bird` 的子類別，你可以把一個 `Duck` 實例指派給標註成 `birdie: Bird` 的參數；但在函式主體裡，檢查器會認定 `birdie.quack()` 這個呼叫不合法，因為 `birdie` 名義上就是隻 `Bird`，而 `Bird` 這個類別沒提供 `.quack()`。執行期實際傳進來的到底是不是 `Duck` 它不管，因為名義型別是靜態把關的——檢查器一行程式都不會跑，它只讀原始碼。這比鴨子型別死板，好處是能在建置流程的早期、甚至你還在 IDE 裡邊打字時，就先把某些臭蟲抓出來。

下面這個有點蠢的小程式，剛好把鴨子型別跟名義型別、以及靜態檢查跟執行期行為，全擺在一起對照：

```
# birds.py
class Bird:
    pass

class Duck(Bird):
    # Duck 是 Bird 的子類別
    def quack(self):
        print('Quack!')

def alert(birdie):
    # 沒貼任何型別提示，檢查器直接略過它
    birdie.quack()

def alert_duck(birdie: Duck) -> None:    # 收一個型別為 Duck 的引數
    birdie.quack()

def alert_bird(birdie: Bird) -> None:    # 收一個型別為 Bird 的引數——問題就藏在這
    birdie.quack()
```

用 Mypy 檢查 `birds.py`，馬上看到一個問題：

```
$ mypy birds.py
birds.py:16: error: "Bird" has no attribute "quack"
Found 1 error in 1 file (checked 1 source file)
```

光讀原始碼，Mypy 就看穿 `alert_bird` 有毛病：型別提示把 `birdie` 宣告成 `Bird`，函式主體卻去呼叫 `birdie.quack()`，而 `Bird` 這個類別根本沒這方法。

接著拿這個模組來用：

```
# daffy.py
from birds import *

daffy = Duck()
alert(daffy)          # 合法，因為 alert 沒型別提示
alert_duck(daffy)     # 合法，alert_duck 要 Duck，而 daffy 正是 Duck
alert_bird(daffy)     # 合法，alert_bird 要 Bird，而 daffy 也是 Bird (Duck 的父類別)
```

對 `daffy.py` 跑 Mypy，它回報的還是 `birds.py` 裡那個老錯誤，至於 `daffy.py` 本身那三個呼叫，它覺得都沒問題。但真的把 `daffy.py` 跑起來，你會看到：

```
$ python3 daffy.py
Quack!
Quack!
Quack!
```

全部跑通！這就是鴨子型別的勝利：執行期 Python 不在乎你宣告成什麼型別，它只認鴨子型別。Mypy 在 `alert_bird` 標了錯，但拿 `daffy` 去呼叫它，執行期卻一點事都沒有。這常常讓 Python 老手第一次碰到時嚇一跳：靜態檢查器有時候會在「我們明知道跑得起來」的程式裡挑出錯。

不過呢，要是幾個月後你被派去擴充這套蠢鳥程式，你大概會慶幸有 Mypy。看看這個同樣用到 `birds` 的模組：

```
# woody.py
from birds import *

woody = Bird()
alert(woody)
alert_duck(woody)
alert_bird(woody)
```

Mypy 檢查 `woody.py` 揪出兩個錯：

```
$ mypy woody.py
birds.py:16: error: "Bird" has no attribute "quack"
woody.py:5: error: Argument 1 to "alert_duck" has incompatible type "Bird"; expected "Duck"
Found 2 errors in 2 files (checked 1 source file)
```

第一個錯在 `birds.py`，就是 `alert_bird` 裡那個 `birdie.quack()`，前面看過了。第二個錯在 `woody.py`：`woody` 是 `Bird` 的實例，所以 `alert_duck(woody)` 不合法，因為那個函式要的是 `Duck`。每隻鴨都是鳥，但不是每隻鳥都是鴨。而這一次，執行期 `woody.py` 裡的呼叫沒有一個會成功，用主控台把這串連環失敗演一遍最清楚：

```
>>> from birds import *
>>> woody = Bird()
>>> alert(woody)
# AttributeError: 'Bird' object has no attribute 'quack' ← Mypy 抓不到，因為 alert 沒型別提示
>>> alert_duck(woody)
# AttributeError: 'Bird' object has no attribute 'quack' ← Mypy 早就警告過：給 alert_duck 的引數型別不對
>>> alert_bird(woody)
# AttributeError: 'Bird' object has no attribute 'quack' ← Mypy 從一開始就說 alert_bird 主體有錯
```

這個小實驗點出：鴨子型別上手容易、也更靈活，但會放任「不支援的操作」拖到執行期才出錯；名義型別則能在執行前就抓到錯，代價是有時會回絕「其實跑得起來」的程式，像剛剛 `alert_bird(daffy)` 那種。不過話說回來，就算它偶爾能跑，`alert_bird` 這函式其實是取錯名字了：它的主體確實需要一個支援 `.quack()` 的物件，而 `Bird` 並沒有。

這個程式裡每個函式都只有一行，但真實程式碼裡它們可能很長，可能把 `birdie` 引數再往下傳給更多函式，而那個 `birdie` 的源頭可能隔著好幾層呼叫，到時候要追出執行期錯誤的根源就很折騰。型別檢查器的價值，就在於讓很多這種錯根本沒機會在執行期發生。

這一節我們從那個沒好好貼標籤的 `double()` 出發，摸清了「型別」跟「操作」在鴨子型別與名義型別裡的關係。接下來就要走一遍替函式貼標籤時最重要的幾種型別；至於 `double()` 該怎麼漂亮地補上型別提示，等後面課程談到「靜態協定」時就會看到。不過在那之前，還有幾個更基礎的型別得先認識。

標註裡用得上的型別

幾乎任何 Python 型別都能拿來當型別提示，只是其中有些限制和建議；此外 `typing` 模組還引進了一些語意偶爾讓人意外的特殊構造。把比方接著用：這一節就是帶你逛一遍「標籤工具箱」裡到底有哪些標籤可貼。會走過的主要型別包括 `typing.Any`、簡單型別與類別、`typing.Optional` 與 `typing.Union`、泛型容器（含 `tuple` 與對映）、抽象基底類別、泛型可迭代物件、參數化泛型與 `TypeVar`、`typing.Protocol`（靜態鴨子型別的關鍵）、`typing.Callable`，最後用 `typing.NoReturn` 收尾。先從一個乍看怪異、好像沒啥用、卻至關重要的型別開始。

Any 型別

漸進式型別系統的核心就是 `Any` 型別，又叫「動態型別」。當檢查器看到一個沒貼標籤的函式：

```
def double(x):  
    return x * 2
```

它會直接當成這樣：

```
def double(x: Any) -> Any:  
    return x * 2
```

意思是 `x` 跟回傳值都能是任何型別，甚至彼此型別還能不一樣，而且 `Any` 被假設成「支援所有可能的操作」。

拿 `Any` 跟 `object` 對照就有意思了。下面這個簽名也收所有型別的引數，因為每種型別都是 `object` 的子型別：

```
def double(x: object) -> object:  
    return x * 2
```

可是檢查器會回絕它，問題出在 `object` 並不支援 `__mul__`：

```
double_object.py:2: error: Unsupported operand types for * ("object" and "int")
```

關鍵在於：越「通用」的型別，介面反而越窄、支援的操作越少。`object` 支援的操作比 `abc.Sequence` 少，`abc.Sequence` 又比 `abc.MutableSequence` 少，後者再比 `list` 少。而 `Any` 是個魔法型別，同時坐在型別階層的最頂端跟最底端：它既是最通用的型別（所以 `n: Any` 收所有型別的值），又是最特化的型別（支援所有可能的操作）——至少在檢查器眼中是這樣理解它的。當然，現實裡沒有任何型別真能支援所有操作，所以一旦用了 `Any`，就等於讓檢查器沒辦法履行它的核心任務：在程式還沒因為執行期例外當掉之前，先抓出可能不合法的操作。

「是...的子型別」與「與...相容」

傳統物件導向的名義型別系統靠的是「是...的子型別」這層關係：給一個類別 `T1` 跟它的子類別 `T2`，那 `T2` 就是 `T1` 的子型別。

```
class T1:
    ...

class T2(T1):
    ...

def f1(p: T1) -> None:
    ...

o2 = T2()
f1(o2)    # OK: 拿子類別實例去填父類別參數沒問題
```

`f1(o2)` 這個呼叫正是「里氏替換原則 (LSP)」的運用。Barbara Liskov 當初就是用「支援的操作」來定義子型別：如果用 `T2` 的物件去頂替 `T1` 的物件、程式照樣正常運作，那 `T2` 就是 `T1` 的子型別。反過來就會違反 LSP：

```
def f2(p: T2) -> None:
    ...
```

```
o1 = T1()
f2(o1)    # 型別錯誤：父類別實例不能拿去填子類別參數
```

從操作角度看很合理：`T2` 身為子類別，繼承且必須支援 `T1` 的所有操作，所以 `T2` 實例能用在任何期望 `T1` 的地方；但反向不一定成立，`T2` 可能多了一些方法，於是 `T1` 實例未必能用在所有期望 `T2` 的地方。這種聚焦在「支援哪些操作」的觀點，也反映在 LSP 的另一個名字——「行為子型別」。

漸進式型別系統裡還多了一層關係叫「與...相容」，它適用於所有「子型別」適用的場合，另外再替 `Any` 開幾條特例。規則有三條：給 `T1` 跟它的子型別 `T2`，`T2` 與 `T1` 相容（這就是里氏替換）；每一種型別都與 `Any` 相容，所以任何型別的物件都能傳給標註成 `Any` 的參數；而 `Any` 也與每一種型別相容，所以你永遠能把 `Any` 型別的物件，放到期望別種型別的地方。後兩條用程式看：

```
def f3(p: Any) -> None:
    ...

o0 = object()
o1 = T1()
o2 = T2()
f3(o0)
f3(o1)
f3(o2)    # 全 OK：規則二

def f4():
    # 沒標回傳型別，隱含就是 Any
    ...

o4 = f4()    # 推斷出來的型別是 Any
f1(o4)
f2(o4)      # 全 OK：規則三
f3(o4)
```

每一套漸進式型別系統，都需要一個像 `Any` 這樣的萬用百搭型別。

簡單型別與類別

像 `int`、`float`、`str`、`bytes` 這種簡單型別，可以直接拿來貼標籤。標準函式庫、外部套件、或你自訂的具體類別（像 `FrenchDeck`、`Vector2d`、`Duck`）也都能用。抽象基底類別在標籤裡同樣好用，等講容器型別、以及後面課程談抽象基底類別時會再回來看。在類別之間，「與...相容」的定義就跟「是...的子型別」一樣：子類別與它的所有父類別相容。不過「實用勝過純粹」，這裡有個重要的例外，留待後面提醒。

Optional 與 Union 型別

前面課程在講「用 `None` 當預設值」時看過 `Optional`，它解決了「拿 `None` 當預設值」的問題：

```
from typing import Optional
def show_count(count: int, singular: str, plural: Optional[str] = None) -> str:
```

`Optional[str]` 其實是 `Union[str, None]` 的縮寫，意思是 `plural` 的型別可以是 `str` 或 `None`。內建的 `ord` 簽名就是個簡單的 `Union`，它收 `str` 或 `bytes`、回傳 `int`：

```
def ord(c: Union[str, bytes]) -> int: ...
```

下面這個函式收 `str`，但可能回傳 `str` 或 `float`：

```
from typing import Union
def parse_token(token: str) -> Union[str, float]:
    try:
        return float(token)
    except ValueError:
        return token
```

能避免的話，盡量別寫「回傳 `Union`」的函式，因為那會逼使用者在執行期還得去檢查回傳值的型別才知道下一步怎麼處理；但在一個簡單運算式求值器的脈絡下，上面這個 `parse_token` 算是合理的用法。`Union[]` 至少要兩個型別，而巢狀的 `Union` 跟攤平的 `Union` 效果一樣，所以 `Union[A, B, Union[C, D, E]]` 等同 `Union[A, B, C, D, E]`。另外 `Union` 對「彼此不相容的型別」才有意義：`Union[int, float]` 是多餘的，因為 `int` 本來就與 `float` 相容，你光標 `float`，它連 `int` 值也會收。

泛型容器

大多數 Python 容器是異質的——一個 list 裡你愛塞什麼型別的混搭都行。但實務上那不太有用：你把東西放進容器，通常是之後要對它們做操作，而那往往代表它們至少得共用某個方法。泛型型別可以用「型別參數」來指定裡頭元素的型別，比方說 list 就能被參數化來約束元素型別：

```
def tokenize(text: str) -> list[str]:    # Python ≥ 3.9
    return text.upper().split()
```

在 Python 3.9 以後，這代表 tokenize 回傳一個「每一項都是 str」的 list。stuff: list 跟 stuff: list[Any] 意思相同：都是「stuff 是一個裝著任意型別物件的 list」。

PEP 585 列出了標準函式庫裡接受泛型標籤的容器。用最簡單的 容器[元素] 形式的，包括 list、set、frozenset、collections.deque，還有 abc.Container、abc.Collection、abc.Sequence、abc.MutableSequence、abc.Set、abc.MutableSet 等。tuple 跟對映型別支援更複雜的標籤，各自留到對應段落講。順帶一提，截至 Python 3.10，都還沒有好辦法替 array.array 標籤——因為它的 typecode 建構引數決定了存的是整數還是浮點數，更難的是要怎麼靜態檢查整數範圍、避免往陣列加元素時在執行期爆 OverflowError（例如 typecode='B' 的陣列只能放 0 到 255 的 int）。目前 Python 的靜態型別系統還挑不起這個擔子。

至於舊版相容（只用 Python 3.9 以後的話可以跳過這段）：在 3.7、3.8，要讓內建容器吃 [] 標記，得先加一行 from __future__ import annotations：

```
from __future__ import annotations    # Python ≥ 3.7
def tokenize(text: str) -> list[str]:
    return text.upper().split()
```

這個 __future__ 匯入在 3.6 以前不管用；要相容到 Python 3.5，得改用 typing 裡的大寫版本：

```
from typing import List                # Python ≥ 3.5
def tokenize(text: str) -> List[str]:
```

```
return text.upper().split()
```

當初為了先撐起泛型標籤，PEP 484 的作者在 `typing` 模組裡造了幾十個泛型型別，對應關係大致是：`list` 對 `typing.List`、`set` 對 `typing.Set`、`frozenset` 對 `typing.FrozenSet`、`collections.deque` 對 `typing.Deque`、`abc.MutableSequence` 對 `typing.MutableSequence`、`abc.Sequence` 對 `typing.Sequence`、`abc.Set` 對 `typing.AbstractSet`、`abc.MutableSet` 對 `typing.MutableSet`。PEP 585 啟動了一個橫跨數年的改善流程，大致四步：先在 3.7 用 `from __future__ import annotations` 讓 `list[str]` 這種寫法能用；在 3.9 把它變成預設，不必再加 `future` 匯入；接著把 `typing` 裡那些多餘的泛型型別標記為「不建議使用」（直譯器不會發警告，而是由檢查器在程式鎖定 3.9 以後的版本時去標）；最後在「3.9 推出五年後的第一個版本」把那些多餘型別移除——照目前的步調，那可能會是 Python 3.14，外號 Python Pi。

tuple 型別

替 tuple 貼標籤有三種路子：當成紀錄、當成具名欄位的紀錄、當成不可變序列。

當成紀錄時，用內建的 `tuple` 並在 `[]` 裡逐一宣告各欄位的型別。比方一個裝著城市名、人口、國名的 tuple `('Shanghai', 24.28, 'China')`，標籤就是 `tuple[str, float, str]`。看一個收經緯度、回傳 Geohash 的函式：

```
shanghai = 31.2304, 121.4737
geohash(shanghai) # 'wtw3sjq6q'
```

```
# coordinates.py
from geolib import geohash as gh # type: ignore ← 這註解叫 Mypy 別抱怨 geolib 沒型別提示
PRECISION = 9

def geohash(lat_lon: tuple[float, float]) -> str: # lat_lon 標成「兩個 float 欄位的 tuple」
    return gh.encode(*lat_lon, PRECISION)
```

欄位很多、或某種 tuple 在你程式裡到處用時，強烈建議改用 `typing.NamedTuple`（前面課程介紹過）：

```
# coordinates_named.py
from typing import NamedTuple
from geolib import geohash as gh # type: ignore
PRECISION = 9

class Coordinate(NamedTuple):
    lat: float
    lon: float

def geohash(lat_lon: Coordinate) -> str:
    return gh.encode(*lat_lon, PRECISION)
```

`typing.NamedTuple` 是「tuple 子類別的工廠」，所以 `Coordinate` 與 `tuple[float, float]` 相容，但反向不成立——畢竟 `Coordinate` 還多了 `NamedTuple` 加上的 `_asdict()` 之類的方法，也可能有自訂方法。實務上這代表：把 `Coordinate` 實例傳給下面這個收 `tuple[float, float]` 的 `display` 是型別安全的：

```
def display(lat_lon: tuple[float, float]) -> str:
    lat, lon = lat_lon
    ns = 'N' if lat >= 0 else 'S'
    ew = 'E' if lon >= 0 else 'W'
    return f'{abs(lat):0.1f}°{ns}, {abs(lon):0.1f}°{ew}'
```

當成「長度不定的不可變序列」時，要指定單一型別，後面接一個逗號跟 `...`（Python 的刪節號記號，是三個句點，不是 Unicode 的 U+2026）。例如 `tuple[int, ...]` 就是「裝著 int 的 tuple」。那個刪節號表示「任意數量、大於等於 1 個元素」都可以，而長度不定的 tuple 沒辦法指定不同型別的欄位。`stuff: tuple[Any, ...]` 跟 `stuff: tuple` 意思相同：長度不定、裝任意型別物件的 tuple。下面這個 `columnize` 就把序列轉成「一列列的表格」，回傳「字串 tuple 的 list」，方便分欄顯示：

```
animals = 'drake fawn heron ibex koala lynx tahr xerus yak zapus'.split()
table = columnize(animals)
```

```
# [('drake', 'koala', 'yak'), ('fawn', 'lynx', 'zapus'), ('heron', 'tahr'), ('ibex', 'xerus')]
```

```
# columnize.py
from collections.abc import Sequence

def columnize(
    sequence: Sequence[str], num_columns: int = 0
) -> list[tuple[str, ...]]:      # 注意回傳型別：字串 tuple 的 list
    if num_columns == 0:
        num_columns = round(len(sequence) ** 0.5)    # 沒指定欄數就抓元素數的平方根
    num_rows, remainder = divmod(len(sequence), num_columns)
    num_rows += bool(remainder)      # 有餘數就多補一列；bool 當 0/1 加上去
    return [tuple(sequence[i::num_rows]) for i in range(num_rows)]
```

泛型對映

泛型對映標成 對映型別[鍵型別, 值型別]。內建的 `dict`，以及 `collections`、`collections.abc` 裡的對映型別，在 Python 3.9 以後都吃這種寫法；更舊的版本就得用 `typing.Dict` 那一掛。下面這個 `name_index` 替 Unicode 字元名稱建一份「倒排索引」，給定起訖字元碼，回傳一個 `dict[str, set[str]]`，把每個單字對應到「名稱裡含有該單字的字元集合」：

```
index = name_index(32, 65)
index['DIGIT'] & index['EIGHT']    # {'8'}：名稱同時含 DIGIT 跟 EIGHT 的字元，就是 '8'
```

```
# charindex.py
import sys
import re
import unicodedata
from collections.abc import Iterator

RE_WORD = re.compile(r'\w+')
STOP_CODE = sys.maxunicode + 1
```

```
def tokenize(text: str) -> Iterator[str]:          # 這是產生器函式，回傳「迭代器」
    """return iterable of uppercased words"""
    for match in RE_WORD.finditer(text):
        yield match.group().upper()

def name_index(start: int = 32, end: int = STOP_CODE) -> dict[str, set[str]]:
    index: dict[str, set[str]] = {}                # 連區域變數都標籤，否則 Mypy 會說它需要型別標註
    for char in (chr(i) for i in range(start, end)):
        if name := unicodedata.name(char, ''):     # 海象運算子：先把名稱指派給 name，再判斷它非空
            for word in tokenize(name):
                index.setdefault(word, set()).add(char)
    return index
```

這段裡頭三個東西是頭一次出現：`tokenize` 是產生器函式（產生器留到後面課程專講）；區域變數 `index` 被標了籤，少了它 Mypy 會要你補；條件式裡用了海象運算子 `:=`，它把 `unicodedata.name(...)` 的結果指派給 `name`，再拿這個結果判斷真假，名稱是空字串就跳過。

抽象基底類別

理想上，函式的參數應該收「抽象型別」（或 3.9 以前對應的 `typing` 版本），而不是具體型別，這樣能給呼叫方更多彈性。看這個簽名：

```
from collections.abc import Mapping
def name2hex(name: str, color_map: Mapping[str, int]) -> str:
```

用 `abc.Mapping`，呼叫方就能塞 `dict`、`defaultdict`、`ChainMap`、`UserDict` 子類別、或任何「是 `Mapping` 子型別」的東西。換成下面這種：

```
def name2hex(name: str, color_map: dict[str, int]) -> str:
```

`color_map` 就只能是 `dict` 或它的子型別（像 `defaultdict`、`OrderedDict`）。特別注意：`collections.UserDict` 的子類別過不了這關，儘管前面課程說過它才是自訂對映的推薦做法——因為 `UserDict` 並不是 `dict` 的子類別，它們是兄弟，都繼承自

`abc.MutableMapping`。所以一般而言，參數標籤用 `abc.Mapping` 或 `abc.MutableMapping` 會比用 `dict` 好。如果 `name2hex` 不需要去改動傳進來的 `color_map`，那 `color_map` 最精準的標籤就是 `abc.Mapping`：這樣呼叫方就不必提供有實作 `setdefault`、`pop`、`update` 的物件（那些是 `MutableMapping` 介面的一部分，不是 `Mapping` 的）。這呼應了 Postel 法則的後半句——「接受時要寬鬆」。

而 Postel 法則前半句也提醒我們「送出時要保守」。函式的回傳值一定是個具體物件，所以回傳標籤就該用具體型別，像前面 `tokenize` 用 `list[str]`：

```
def tokenize(text: str) -> list[str]:
    return text.upper().split()
```

Python 文件在 `typing.List` 條目下就說：它是 `list` 的泛型版，適合用來標回傳型別；要標引數，則偏好用 `Sequence`、`Iterable` 這類抽象容器型別。`typing.Dict`、`typing.Set` 條目也有類似說明。記得從 Python 3.9 起，`collections.abc` 的多數抽象基底類別、`collections` 裡的具體類別、還有內建容器，都支援像 `collections.deque[str]` 這種泛型寫法；對應的 `typing` 版本只有在要相容 3.8 以前的程式時才需要。

至於數值那塊：`numbers` 套件定義了所謂的「數值塔」，是一個線性的抽象基底類別階層，由上而下是 `Number`、`Complex`、`Real`、`Rational`、`Integral`。這些抽象基底類別拿來做執行期型別檢查運作良好，但靜態型別檢查並不支援它們——PEP 484 否決了 `numbers` 那套抽象基底類別，改規定把內建的 `complex`、`float`、`int` 當特例處理（也就是前面課程說過的「`int` 與 `complex` 相容」）。後面課程在對比協定與抽象基底類別時會再回來談這題。實務上，要替數值引數標靜態型別，大致有幾條路：用具體的 `int`、`float`、`complex`；宣告像 `Union[float, Decimal, Fraction]` 這種聯集；或者不想把具體型別寫死，就用像 `SupportsFloat` 這類數值協定，不過那得先懂後面課程要講的「靜態協定」。

Iterable

剛引述的那段文件就建議「引數用 `Sequence` 或 `Iterable`」。標準函式庫裡 `math.fsum` 的 `Iterable` 引數就是個好示意：

```
def fsum(__seq: Iterable[float]) -> float:    # 開頭雙底線是慣例，表示這是僅限位置的引數
```

再看一個用 `Iterable` 參數、而它產出的項目是 `tuple[str, str]` 的函式：

```
l33t = [('a', '4'), ('e', '3'), ('i', '1'), ('o', '0')]
text = 'mad skilled noob powned leet'
zip_replace(text, l33t)  # 'm4d sk1ll3d n00b p0wn3d l33t'
```

```
# replacer.py
from collections.abc import Iterable

FromTo = tuple[str, str]  # 型別別名：把 tuple[str, str] 取名叫 FromTo，簽名好讀很多

def zip_replace(text: str, changes: Iterable[FromTo]) -> str:
    for from_, to in changes:  # changes 是 Iterable[FromTo]，等同 Iterable[tuple[str, str]]
        text = text.replace(from_, to)
    return text
```

最後談 `abc.Iterable` 跟 `abc.Sequence` 怎麼選。用 `Iterable` 當參數的好處是，呼叫方可以餵一個「邊產邊吃」的可迭代物件，而不必先建好一整個序列，當輸入量很大時可能省下大量記憶體。但反過來，剛才那個 `columnize` 就非得收 `Sequence` 不可、不能只收 `Iterable`，因為它得先對輸入取 `len()` 來算出列數。跟 `Sequence` 一樣，`Iterable` 最適合當參數型別，拿來當回傳型別則太籠統了——函式對自己回傳的具體型別應該講得更精準。和 `Iterable` 關係很近的是 `Iterator` 型別，剛剛 `charindex.py` 就拿它當回傳型別，這個會留到後面講產生器與經典迭代器的課程再細談。

參數化泛型與 `TypeVar`

把比方接著用：`TypeVar`（型別變數）就像一台影印機的模具——你放什麼型別的東西進去，它就照著那個型別印出來，讓「參數的型別」能反映到「結果的型別」上。所謂「參數化泛型」，就是寫成 `list[T]` 這種，`T` 是個型別變數，每次使用時會被綁到某個具體型別。

下面這個 `sample` 收一個「元素型別為 `T` 的序列」跟一個 `int`，回傳一個「同樣是 `T` 型別、從第一個引數裡隨機挑出來」的 `list`：

```
# sample.py
from collections.abc import Sequence
from random import shuffle
from typing import TypeVar
```

```
T = TypeVar('T')

def sample(population: Sequence[T], size: int) -> list[T]:
    if size < 1:
        raise ValueError('size must be >= 1')
    result = list(population)
    shuffle(result)
    return result[:size]
```

之所以要用型別變數：如果你拿一個 `tuple[int, ...]`（它與 `Sequence[int]` 相容）來呼叫它，型別參數就是 `int`，於是回傳型別就是 `list[int]`——進去是 `int`，出來就是 `int` 的 `list`。標準函式庫的 `statistics.mode` 也是個好例子，它回傳資料裡出現最多次的那一項，像 `mode([1, 1, 2, 3, 3, 3, 3, 4])` 會回 `3`。

不用 `TypeVar` 的話，`mode` 大概會長這樣：

```
# mode_float.py
from collections import Counter
from collections.abc import Iterable

def mode(data: Iterable[float]) -> float:
    pairs = Counter(data).most_common(1) # most_common(1) 抓出現最多次的那一組（值，次數）
    if len(pairs) == 0:
        raise ValueError('no mode for empty data')
    return pairs[0][0]
```

`mode` 很多時候處理的是 `int` 或 `float`，但 Python 還有別的數值型別，而我們會希望「回傳型別跟著傳入元素的型別走」。用 `TypeVar` 可以改善，先看一個簡單但有問題的版本：

```
from collections.abc import Iterable
from typing import TypeVar
```

```
T = TypeVar('T')

def mode(data: Iterable[T]) -> T:
    ...
```

`T` 第一次出現在簽名裡時可以是任何型別，第二次出現就代表跟第一次同一個型別。問題是這樣一來，每個可迭代物件都與 `Iterable[T]` 相容，包括那些「裝著不可雜湊型別、`collections.Counter` 根本處理不了」的可迭代物件。我們得限制 `T` 能被指派成哪些型別，接下來兩段就是兩種做法。

限制式 TypeVar

`TypeVar` 可以多收幾個位置引數來限制型別參數。我們可以讓 `mode` 只收特定的數值型別：

```
from collections.abc import Iterable
from decimal import Decimal
from fractions import Fraction
from typing import TypeVar

NumberT = TypeVar('NumberT', float, Decimal, Fraction)  # T 只能是這三種之一

def mode(data: Iterable[NumberT]) -> NumberT:
    ...
```

這比之前好，這也曾是 `typedshd` 上 `statistics.pyi` 在 2020 年 5 月 25 日對 `mode` 的簽名。可是官方文件裡還有這個用法：

`mode(["red", "blue", "blue", "red", "green", "red", "red"])` 會回 `'red'`。急就章的話，我們可以把 `str` 加進去：`NumberT = TypeVar('NumberT', float, Decimal, Fraction, str)`。這當然能動，但 `NumberT` 連 `str` 都收就名不副實了，更要命的是我們不可能每發現 `mode` 能處理一種新型別就無止盡地往清單裡加。`TypeVar` 還有另一招更好用。

有上界的 TypeVar

回頭看 `mode` 的主體，會發現它用 `Counter` 來計次排名，而 `Counter` 是建在 `dict` 上的，所以 `data` 的元素型別必須是「可雜湊的」。乍看下面這簽名好像可行：

```
from collections.abc import Iterable, Hashable

def mode(data: Iterable[Hashable]) -> Hashable:
    ...
```

但問題來了：回傳項目的型別變成 `Hashable` —— 這個抽象基底類別只實作了 `__hash__`，於是檢查器除了讓你對回傳值呼叫 `hash()` 之外什麼都不准做，沒什麼用。解法是 `TypeVar` 的另一個選用參數 `bound`，它替可接受的型別設一個「上界」：

```
# mode_hashable.py
from collections import Counter
from collections.abc import Iterable, Hashable
from typing import TypeVar

HashableT = TypeVar('HashableT', bound=Hashable) # T 可以是 Hashable 或它的任何子型別

def mode(data: Iterable[HashableT]) -> HashableT:
    pairs = Counter(data).most_common(1)
    if len(pairs) == 0:
        raise ValueError('no mode for empty data')
    return pairs[0][0]
```

兩者的差別一句話講完：限制式型別變數會被設成「`TypeVar` 宣告裡點名的那幾種型別之一」；有上界的型別變數則會被設成「運算式推斷出來的型別」，只要那個推斷型別與 `bound=` 宣告的界線相容就行。`typing.TypeVar` 還有 `covariant`、`contravariant` 兩個選用參數，留到後面課程談變異性時再講。

預先定義好的 `AnyStr`

`typing` 模組內附一個預先定義好的型別變數叫 `AnyStr`，它定義成 `AnyStr = TypeVar('AnyStr', bytes, str)`，用在許多「收 `bytes` 或 `str`、並回傳同型別值」的函式上。接著進到 `typing.Protocol`，這是 Python 3.8 的新功能，能支援更 Pythonic 的型別提示用法。

靜態協定

`Protocol` 型別（出自 PEP 544）類似 Go 語言裡的介面：你指定一個或多個方法來定義一個協定型別，檢查器就會去驗證「需要該協定的地方，那些方法有沒有被實作」。在 Python 裡，協定定義寫成 `typing.Protocol` 的子類別；但實作該協定的類別，不需要繼承、註冊、或宣告任何跟「定義協定那個類別」的關係——要不要採用協定、有沒有照辦，全交給檢查器去判定。

來看一個靠 `Protocol` 加 `TypeVar` 解決的問題。假設你要寫一個 `top(it, n)`，回傳可迭代物件 `it` 裡最大的 `n` 個元素：

```
top([4, 1, 5, 2, 6, 7, 3], 3)           # [7, 6, 5]
l = 'mango pear apple kiwi banana'.split()
top(l, 3)                               # ['pear', 'mango', 'kiwi']
l2 = [(len(s), s) for s in l]
top(l2, 3)                              # [(6, 'banana'), (5, 'mango'), (5, 'apple')]
```

參數化的 `top` 會像這樣，但 `T` 還沒約束：

```
def top(series: Iterable[T], length: int) -> list[T]:
    ordered = sorted(series, reverse=True)
    return ordered[:length]
```

問題在於 `T` 要怎麼約束？它不能是 `Any` 或 `object`，因為 `series` 得餵得進 `sorted`。 `sorted` 表面上收 `Iterable[Any]`，那是因為它的選用參數 `key` 能替每個元素算出排序鍵。要是你給 `sorted` 一串純 `object`、又不給 `key`，會怎樣？

```
l = [object() for _ in range(4)]
sorted(l)
# TypeError: '<' not supported between instances of 'object' and 'object'
```

錯誤訊息透露：`sorted` 是對元素動用 `<` 運算子。是不是只要會 `<` 就行？再做個小實驗：

```
class Spam:
    def __init__(self, n):
        self.n = n
    def __lt__(self, other):      # 實作 < 運算子背後的方法
        return self.n < other.n
    def __repr__(self):
        return f'Spam({self.n})'

l = [Spam(n) for n in range(5, 0, -1)]  # [Spam(5), Spam(4), Spam(3), Spam(2), Spam(1)]
sorted(l)                             # [Spam(1), Spam(2), Spam(3), Spam(4), Spam(5)]
```

證實了：`Spam` 的 list 能排序，因為它實作了支援 `<` 的 `__lt__`。所以 `T` 該被限制成「有實作 `__lt__` 的型別」。前面那個 `mode` 我們需要的是「實作 `__hash__`」，剛好有現成的 `typing.Hashable` 能當上界；但這次 `typing` 跟 `abc` 裡都沒有合適的型別，只好自己造一個協定：

```
# comparable.py
from typing import Protocol, Any

class SupportsLessThan(Protocol):      # 協定就是 typing.Protocol 的子類別
    def __lt__(self, other: Any) -> bool: ...  # 協定主體放一個以上的方法定義，主體用 ... 帶過
```

一個型別 `T` 與協定 `P` 相容的條件是：`T` 實作了 `P` 裡定義的所有方法，而且型別簽名對得上。有了 `SupportsLessThan`，就能定義能正常運作的 `top`：

```
# top.py
from collections.abc import Iterable
from typing import TypeVar
from comparable import SupportsLessThan
```

```
LT = TypeVar('LT', bound=SupportsLessThan)    # T 必須與「會 < 這招」的協定相容
```

```
def top(series: Iterable[LT], length: int) -> list[LT]:  
    ordered = sorted(series, reverse=True)  
    return ordered[:length]
```

用 `pytest` 試開一下。下面這份測試先用一個會吐 `tuple[int, str]` 的產生器運算式去呼叫 `top`，再用一串 `object` 去呼叫——後者預期會引發 `TypeError`：

```
# top_test.py (部分)  
from collections.abc import Iterator  
from typing import TYPE_CHECKING  
import pytest  
from top import top  
  
def test_top_tuples() -> None:  
    fruit = 'mango pear apple kiwi banana'.split()  
    series: Iterator[tuple[int, str]] = (          # 明確宣告 series 型別，讓 Mypy 輸出好讀  
        (len(s), s) for s in fruit)  
    length = 3  
    expected = [(6, 'banana'), (5, 'mango'), (5, 'apple')]  
    result = top(series, length)  
    if TYPE_CHECKING:                             # 這個 if 在執行時擋住下面三行不跑  
        reveal_type(series)  
        reveal_type(expected)  
        reveal_type(result)  
    assert result == expected  
  
# 故意製造型別錯誤  
def test_top_objects_error() -> None:  
    series = [object() for _ in range(4)]  
    if TYPE_CHECKING:  
        reveal_type(series)
```

```
with pytest.raises(TypeError) as excinfo:
    top(series, 3)                # 這行會被 Mypy 標成錯
assert "'<' not supported" in str(excinfo.value)
```

這裡有幾個關鍵：`typing.TYPE_CHECKING` 這個常數在執行期永遠是 `False`，但檢查器做檢查時會假裝它是 `True`；`reveal_type()` 不能在執行期呼叫，它不是真函式，而是 Mypy 的除錯工具，所以沒有匯入它——Mypy 會替每個 `reveal_type()` 呼叫吐一行除錯訊息，顯示推斷出的型別。對這份測試跑 Mypy，可以看到 `TypeVar` 確實照預期運作：

```
$ mypy top_test.py
top_test.py:32: note: Revealed type is "typing.Iterator[Tuple[builtins.int, builtins.str]]"
top_test.py:34: note: Revealed type is "builtins.list[Tuple[builtins.int, builtins.str]]"
top_test.py:41: note: Revealed type is "builtins.list[builtins.object*]"
top_test.py:43: error: Value of type variable "LT" of "top" cannot be "object"
Found 1 error in 1 file (checked 1 source file)
```

`reveal_type(series)` 顯示它是我明確宣告的 `Iterator[tuple[int, str]]`；`reveal_type(result)` 確認 `top` 回傳的型別正是我要的——給定 `series` 的型別，結果是 `list[tuple[int, str]]`。在第二個測試裡，`series` 被推斷成 `list[object*]`（Mypy 會在「推斷出來、而非我親手標註」的型別後面加個 `*`）；最後 Mypy 標出這個測試故意觸發的錯：可迭代物件 `series` 的元素型別不能是 `object`，它必須是 `SupportsLessThan` 型別。

協定型別比抽象基底類別強的一個關鍵在於：一個型別不需要任何特別宣告，就能與協定相容。這讓你能拿既有型別、甚至是你管不到的程式碼裡的型別來湊。我不必把 `str`、`tuple`、`float`、`set` 去繼承或註冊到 `SupportsLessThan`，就能在「期望 `SupportsLessThan` 參數」的地方用它們，它們只要有實作 `__lt__` 就好。而檢查器照樣辦得了事，因為 `SupportsLessThan` 是被明確定義成 `Protocol`——這跟鴨子型別常見的那種「隱含協定」恰成對比，後者對檢查器是隱形的。這也就是為什麼這功能叫「靜態鴨子型別」：替 `top` 的 `series` 參數貼標籤的解法，等於是說「`series` 名義上是什麼型別不重要，只要它實作了 `__lt__`」。Python 的鴨子型別一直都能隱含地這麼說，卻把靜態檢查器晾在一旁摸不著頭緒；現在我們能把鴨子型別「明說」給檢查器聽了。關於 `typing.Protocol` 還有更多細節，後面課程在對比結構型別、鴨子型別與抽象基底類別，以及講多載簽名時會再回來談。

Callable

要替「回呼參數」或「高階函式回傳的可呼叫物件」貼標籤，`collections.abc` 模組提供了 `Callable` 型別（還沒用 3.9 的人可以從 `typing` 取用）。它的參數化長這樣：`Callable[[參數型別1, 參數型別2], 回傳型別]`，其中那串參數清單可以有零個到多個型別。看一個 `repl` 函式的用法，它是某個簡單互動式直譯器的一部分（後面課程會看到）：

```
def repl(input_fn: Callable[[Any], str] = input) -> None:
    ...
```

平常 `repl` 用 Python 內建的 `input` 來讀使用者輸入的運算式；但為了自動化測試、或接其他輸入來源，它收一個選用的 `input_fn`——一個「參數型別和回傳型別都跟 `input` 一樣」的可呼叫物件。`input` 在 `typeshed` 上的簽名是 `def input(__prompt: Any = ...) -> str:`，它與 `Callable[[Any], str]` 這個標籤相容。要注意沒有語法能標「選用引數或關鍵字引數的型別」，文件也說這類函式型別很少拿來當回呼型別用。如果你需要一個能對上「彈性簽名函式」的標籤，就把整串參數清單換成 `...`，寫成 `Callable[..., 回傳型別]`。

泛型型別參數跟型別階層的互動，帶出一個新概念：變異性。想像一個溫控系統，有個 `update` 函式：它呼叫 `probe` 拿目前溫度，再呼叫 `display` 把溫度秀給使用者（這裡刻意把兩者當引數傳進去，是為了教學對照——一個看回傳型別、一個看參數型別）：

```
from collections.abc import Callable

def update(
    probe: Callable[[], float],          # probe: 不收引數、回傳 float 的可呼叫物件
    display: Callable[[float], None],    # display: 收一個 float、回傳 None
) -> None:
    temperature = probe()
    # 想像這裡有一大堆控制程式碼
    display(temperature)

def probe_ok() -> int:
    return 42

def display_wrong(temperature: int) -> None:
    print(hex(temperature))
```

```

update(probe_ok, display_wrong)          # 型別錯誤

def display_ok(temperature: complex) -> None:
    print(temperature)

update(probe_ok, display_ok)             # OK

```

`probe_ok` 與 `Callable[[], float]` 相容，因為「回傳 `int`」不會弄壞「期望拿到 `float`」的程式碼。但 `display_wrong` 與 `Callable[[float], None]` 不相容，因為沒人能保證「一個期望 `int` 的函式」處理得了 `float`——例如 Python 的 `hex` 收 `int` 卻拒收 `float`，所以 Mypy 會標那一行錯。反過來 `display_ok` 與 `Callable[[float], None]` 相容，因為「一個能收 `complex` 的函式」當然也應付得了 `float` 引數，這行 Mypy 就放行。

歸納一下：當程式期望「回傳 `float` 的回呼」時，你給它「回傳 `int` 的回呼」沒問題，因為 `int` 值永遠能用在期望 `float` 的地方。正式講法是：`Callable[[], int]` 是 `Callable[[], float]` 的子型別——方向跟「`int` 是 `float` 的子型別」一致，所以說 `Callable` 在「回傳型別」上是協變的。但在參數那一側，當程式要的是「能處理 `float` 的回呼」時，你給它「收 `int` 的回呼」卻是型別錯誤：雖然 `int` 是 `float` 的子型別，但在參數化的 `Callable` 裡關係反轉了——`Callable[[float], None]` 才是 `Callable[[int], None]` 的子型別，所以說 `Callable` 在「宣告的參數型別」上是逆變的。關於不變、協變、逆變的更多細節和說明，留到後面課程談變異性時再展開。

NoReturn

這個特殊型別只用在「永遠不會回傳的函式」的回傳型別標註上。比方 `sys.exit()` 會引發 `SystemExit` 來終止 Python 行程，它在 `typeshed` 上的簽名是 `def exit(__status: object = ...) -> NoReturn: ...`。其中 `__status` 是僅限位置的參數、有預設值；`stub` 檔不會把預設值寫出來，而是用 `...` 代替。`__status` 的型別是 `object`，代表它也可以是 `None`，所以再特別標 `Optional` 就多餘了。後面課程裡有個 `_flag_unknown_attrs` 方法也用到 `NoReturn`，那個方法會先湊出一段親切又完整的錯誤訊息、然後引發例外。這個漫長課程的最後一節，要談的是位置參數與可變參數。

替「僅限位置參數」與「可變參數」貼標籤

還記得前面課程那個 `tag` 函式嗎？它的簽名上次出現在講「僅限位置參數」時長這樣：

```

def tag(name, /, *content, class_=None, **attrs):

```

把它完整貼上標籤、並拆成好幾行寫（長簽名常見的排法，換行方式就照 `blue` 這個格式化工具會做的樣子）：

```
from typing import Optional

def tag(
    name: str,
    *content: str,
    class_: Optional[str] = None,
    **attrs: str,
) -> str:
    ...
```

接著用先前的分流比方來看那兩個「袋子」。注意可變位置參數的標籤 `*content: str`：它代表「打包配菜的那個袋子」裡，每一樣引數都得是 `str`；而在函式主體裡，區域變數 `content` 的型別會是 `tuple[str, ...]`。至於可變關鍵字引數，這裡標的是 `**attrs: str`，所以函式內 `attrs` 的型別會是 `dict[str, str]`；要是改標 `**attrs: float`，`attrs` 在函式裡就會是 `dict[str, float]`。如果 `attrs` 必須收不同型別的值，那就得用 `Union[]` 或 `Any`，寫成 `**attrs: Any`。

僅限位置參數的 `/` 記法，只在 Python 3.8 以後才有；在 3.7 或更早的版本，那會是語法錯誤。PEP 484 的慣例是改在「每個僅限位置參數」的名字前面加兩個底線——等於替「這道只能照排隊順序點、不能指名點」做個記號。同一個 `tag` 簽名，改用兩行、走 PEP 484 慣例就是這樣：

```
from typing import Optional

def tag(__name: str, *content: str, class_: Optional[str] = None,
        **attrs: str) -> str:
    ...
```

這兩種「宣告僅限位置參數」的寫法，Mypy 都看得懂、也都會把關。替這個漫長課程收尾之前，接下來會簡短看看型別提示、以及它所支撐的那套靜態型別系統，究竟有哪些極限。

不完美的型別、紮實的測試

維護大型企業程式碼庫的人回報說，靜態型別檢查器確實揪出不少臭蟲，而且修起來比「等程式上了正式環境才發現」便宜得多。不過有件事很重要得說清楚：就我所知的那些公司來說，在靜態型別被引進之前，自動化測試早就是標準做法、而且普遍採用了。

就算在型別檢查最能發揮的場合，它也不能被當成「正確性的最終裁判」。你很容易就會碰到兩種狀況：一種是偽陽性，工具對「其實正確的程式」報了型別錯誤；另一種是偽陰性，工具對「其實有錯的程式」卻悶不吭聲不報錯。

而且，如果硬要逼自己對所有東西都做型別檢查，反而會犧牲掉 Python 的一部分表達力。有些很順手的功能根本沒辦法被靜態檢查，像是 `config(**settings)` 這種引數拆解；還有屬性（property）、描述器、元類別、以及一般的元程式設計這些進階功能，檢查器要不是支援得很差，就是根本理解不來；更別說型別檢查器總是落在 Python 版本後面，遇到用了新語言特性的程式，常常回絕、甚至直接當掉，有些情況一拖就是一年多。

再來，很多常見的資料限制——連很簡單的那種——也沒辦法用型別系統表達。比方說，型別提示沒辦法保證「數量必須是大於 0 的整數」，或「標籤必須是 6 到 12 個 ASCII 字母組成的字串」。整體來看，要抓商業邏輯裡的錯，型別提示幫不上什麼忙。

把這些但書擺在一起就會明白：型別提示不可能是軟體品質的台柱；要是毫無例外地強制使用它，只會把上面那些缺點放得更大。比較務實的看法是，把靜態型別檢查器當成現代 CI 流水線上的「其中一道工具」，跟測試執行器、程式碼檢查工具（linter）等等並列。CI 流水線的目的是降低軟體出包，而自動化測試能抓到一大堆型別提示力有未逮的臭蟲。一句話：只要是你寫得出來的 Python 程式，你就測得了它——不管它有沒有貼型別提示。

到這裡，目前對 Python 型別提示的介紹就告一段落了。型別提示也是後面課程的重點，那裡會談泛型類別、變異性、多載簽名、型別轉換等等；在這之前，它還會在接下來課程的不少地方客串露面。

第三篇 裝飾器與閉包

函式裝飾器（function decorator）的作用，是讓我們可以在原始碼裡頭幫某個函式「貼個標記」，藉此強化它原本的行為。這東西威力很強，但要真的玩得透，前提是你得先搞懂閉包（closure）——所謂閉包，就是當一個函式去抓取了「定義在它自己身體外面」的變數時，所形成的那個東西。

你可以把它想成這樣：有個師傅在某間老店學了一手私房醬料的配方，後來自己出來開店，那間老店都收掉了，可是他煮出來的味道還是一模一樣，因為那個配方早就跟著他走、被他「記」在身上了。閉包就是這種感覺——函式被建立出來的時候，會把它「出生環境」裡用到的

變數一起打包帶走，就算那個環境之後消失了，函式照樣記得、照樣能用。

在 Python 裡最冷門、最少人碰的保留字就是 `nonlocal`，它是從 Python 3.0 才開始加進來的。老實講，如果你寫程式一輩子都乖乖待在「以類別為中心的物件導向」這條路上，那你大概到老都用不到它，照樣可以過得很爽、寫得很順。可是一旦你想自己動手做函式裝飾器，你就非得把閉包搞懂不可，而到了那個時候，你自然就會明白為什麼會需要 `nonlocal` 這個關鍵字——它的存在不是來搞你的，是來解決一個你閃不掉的問題。

除了用在裝飾器上面以外，閉包在另外兩個地方也很關鍵：一個是只要你的寫法牽涉到回呼（callback），閉包幾乎是跑不掉的；另一個是當你判斷情況適合、想用函數式（functional）的風格來寫程式的時候，它一樣是基本功。

這門課最終要帶你弄清楚的，就是函式裝飾器到底是怎麼運作的，從最單純的「註冊型裝飾器」一路講到比較複雜的「帶參數的裝飾器」。不過在走到那一步之前，我們得先把幾件事鋪好底：Python 是怎麼去解讀裝飾器這個語法的、Python 又是憑什麼決定某個變數到底算不算區域變數（local）、閉包為什麼會存在以及它實際上是怎麼運作的，還有 `nonlocal` 究竟是來解決什麼樣的麻煩。

把這些地基都打穩之後，我們才有辦法去處理後面更進階的裝飾器主題：怎麼寫出一個「行為端正、不會出包」的裝飾器、標準函式庫裡那幾個很猛的現成裝飾器（像是 `functools` 裡的 `@cache`、`@lru_cache` 跟 `@singledispatch`），以及最後——怎麼自己實作一個帶參數的裝飾器。

裝飾器入門（Decorators 101）

裝飾器（decorator）說穿了，就是一個「可呼叫的東西」（callable），它會把另一個函式當成參數收進去——那個被收進去的，就叫做「被裝飾的函式」。裝飾器拿到這個函式之後，可能會對它做點手腳、做些處理，然後再回傳一個函式出來。

你可以這樣想：裝飾器就像一個「替身仲介」。你把本尊 `target` 送進去，仲介搞不好幫你換了一個分身回來，以後別人喊 `target`，跑出來辦事的其實是那個分身，本尊早就被掉包了。重點是——外面的人根本不知道被換過，他們照樣喊同一個名字。

先看語法這層。假設手邊已經有一個叫 `decorate` 的裝飾器，下面這段：

```
@decorate          # 在 target 頭上貼一張「decorate」的標籤
def target():       # 定義一個叫 target 的函式
    print('running target()')
```

它跟你親手寫成下面這樣，效果是一模一樣的：

```
def target():          # 先正常定義 target
    print('running target()')

target = decorate(target)  # 把 target 丟給 decorate，再把回傳的東西重新綁回 target 這個名字
```

兩段跑完的最終狀態完全相同：到最後，`target` 這個名字會被綁到「`decorate(target)` 回傳的那個函式」身上。這裡有個很容易被忽略的眉角——回傳的那個，有可能還是原本那個 `target`，但也很有可能是「換過的另一個函式」。`@decorate` 這個寫法，本質上就只是 `target = decorate(target)` 的縮寫而已，沒有別的魔法。

接著用一段實際在直譯器裡敲出來的對話，來確認「函式真的被掉包了」這件事：

```
>>> def deco(func):          # 定義裝飾器 deco，收進來的 func 就是待會要被裝飾的函式
...     def inner():          # 在 deco 內部又定義了一個 inner 函式（這就是那個「分身」）
...         print('running inner()')
...     return inner          # 注意：deco 回傳的是 inner 這個函式物件本身，沒有加括號去呼叫它
...
>>> @deco                    # 用 deco 來裝飾下面的 target
... def target():
...     print('running target()')
...
>>> target()                  # 呼叫 target，照理說該印 "running target()"
running inner()                # 結果卻印出 "running inner()" ——本尊被分身掉包了
>>> target                    # 直接看 target 現在到底是誰
<function deco.<locals>.inner at 0x10063b598> # 真相大白：target 現在指向的是 inner
```

把這段拆開來看就清楚了。`deco` 回傳的是它內部的 `inner` 函式物件；`target` 被 `deco` 裝飾過後，呼叫 `target` 實際上跑的根本是 `inner`；最後直接檢查 `target`，會發現它已經變成對 `inner` 的參照（reference）了。那串 `deco.<locals>.inner` 的意思就是「`inner` 這個函式，是住在 `deco` 內部區域裡的」。

嚴格講起來，裝飾器其實只是一種「語法糖」(syntactic sugar)。就像剛剛看到的，你大可不必用 `@` 這個寫法，直接把裝飾器當成一般的可呼叫物件來呼叫、手動把另一個函式餵給它，效果完全一樣。有時候這樣手動呼叫反而更方便，特別是在做元程式設計 (metaprogramming)、也就是在程式執行的當下去動態改變它行為的那種情境。

關於裝飾器，有三件事抓住了就等於抓住了精髓：第一，裝飾器是一個函式、或是其他任何可呼叫的東西；第二，裝飾器有可能把被裝飾的函式，整個換成另一個函式；第三，裝飾器是在「模組被載入的當下」就立刻執行的。

這三點裡頭，第三點特別關鍵，也是接下來這門課要好好聚焦進去的地方。

順帶補一句更實務、但邏輯完全一樣的做法：你大概有發現，上面 `target` 被掉包成 `inner` 之後，連名字、身分都跟著變了——別人去查它叫什麼，看到的會是 `inner` 而不是 `target`。所以正規一點的寫法，會在 `inner` 上面再加一道 `@functools.wraps(func)`，讓分身去「冒用」本尊的名字與相關資料，這樣掉包歸掉包，至少對外看起來名字還是對的。這個技巧後面講到「行為端正的裝飾器」時還會再深入，現在先有個印象就好。

Python 何時執行裝飾器 (When Python Executes Decorators)

裝飾器有一個很關鍵的脾氣：它是在「被裝飾的函式一定義完」的當下就馬上跑起來，而這個時間點通常落在匯入時 (import time)，也就是 Python 載入某個模組的那一刻。這跟函式本身要等到「被呼叫」才動起來，是兩件完全不同的事。

打個比方你就懂了。想像有一間公司，門口擺了一本簽到簿。員工只要一進門，警衛就立刻拉著他在簽到簿上簽名報到——這個「簽到」動作，是進門當下就發生的；但簽完到不代表他馬上開始幹活，真正開始做事，得等到主管點名叫他。裝飾器跑起來，就是那個「一進門就簽到」的瞬間；而被裝飾的函式真正執行，是後面「被點名」才發生的。

來看一支叫 `registration.py` 的模組：

```
registry = [] # registry 用來保存「被 @register 貼過標籤」的函式參照，一開始是空的

def register(func):
    print(f'running register({func})') # register 收進一個函式當參數
    registry.append(func)             # 印出現在正在裝飾的是哪個函式（純粹給我們看）
    return func                       # 把這個 func 塞進 registry 裡（這就是「簽到」）
                                     # 一定要回傳一個函式；這裡原封不動把收到的那個還回去

@register                             # f1 被 @register 裝飾
```

```

def f1():
    print('running f1()')

@register                # f2 也被 @register 裝飾
def f2():
    print('running f2()')

def f3():                # f3 沒貼標籤，不裝飾（這個員工不用簽到）
    print('running f3()')

def main():              # main 負責統籌：先秀出簽到簿，再依序點名 f1、f2、f3
    print('running main()')
    print('registry ->', registry)
    f1()
    f2()
    f3()

if __name__ == '__main__': # 只有當 registration.py 是被「當成主程式直接跑」時，才呼叫 main()
    main()

```

這裡 `register` 的回傳值要特別留意：它回傳的是「收到的同一個函式」，原樣奉還。所以 `f1`、`f2` 被裝飾之後，名字綁的其實還是它們本來自己——這跟前一課那種會掉包成分身的裝飾器不一樣，這支 `register` 的目的不是換貨，而是「順手把你登記進簽到簿」而已。

把這支檔案當成主程式直接跑，輸出會長這樣：

```

$ python3 registration.py
running register(<function f1 at 0x100631bf8>)
running register(<function f2 at 0x100631c80>)
running main()
registry -> [<function f1 at 0x100631bf8>, <function f2 at 0x100631c80>]
running f1()

```

```
running f2()
running f3()
```

仔細看執行順序，你會發現 `register` 整整跑了「兩次」，而且是在模組裡其他任何函式都還沒動之前就先跑完了。`register` 每被呼叫一次，手上接到的就是「那個被裝飾的函式物件」，像是 `<function f1 at 0x100631bf8>` 這種東西。等到整個模組載入完畢，`registry` 這個串列裡頭就已經乖乖躺著 `f1` 跟 `f2` 兩個函式的參照了。至於這兩個——加上沒被裝飾的 `f3`——它們本身要等到 `main` 裡頭明確點名呼叫，才會真正執行印出東西。

再看另一種情況：如果 `registration.py` 不是被直接跑，而是被別人 `import` 進去：

```
>>> import registration
running register(<function f1 at 0x10063b1e0>)
running register(<function f2 at 0x10063b268>)
```

你會發現，光是 `import` 這一個動作，`register` 就已經自動跑了兩次，`f1` 跟 `f2` 早就完成簽到了。這時候你去翻簽到簿來看：

```
>>> registration.registry
[<function f1 at 0x10063b1e0>, <function f2 at 0x10063b268>]
```

裡面果然已經有 `f1` 跟 `f2` 了。注意這時候 `main` 完全沒被呼叫（因為不是當主程式跑），所以那些 `running f1()` 之類的字一個都沒出現——簽到歸簽到，沒人點名，大家就都還在位子上坐著。

這整段最想敲進你腦袋裡的重點就是：函式裝飾器在「模組一被匯入」的當下就立刻執行了，但被裝飾的函式卻要等到「明確被呼叫」時才會跑。這正好點出 Python 圈子裡常掛在嘴邊的兩個概念——匯入時（`import time`）跟執行時（`runtime`）——之間的差別。

最後補一個邏輯相同、但更實務一點的觀念：這種「用裝飾器自動把東西登記進一份清單」的手法，在真實的外掛（`plugin`）系統裡非常常見，等於是讓每個模組一被載入，就自動把自己的功能掛上去，主程式完全不用一個一個手動去寫死。如果你擔心同一個函式不小心被登記到兩次，把 `registry` 從串列（`list`）換成集合（`set`），邏輯一樣、但能順便幫你擋掉重複——這是同一套思路下很自然的小升級。

註冊型裝飾器（Registration Decorators）

如果拿真實世界裡裝飾器常見的用法來對照，上一堂課那支 `registration.py` 其實有兩個地方不太典型。第一個是，它把「裝飾器函式」跟「被裝飾的那些函式」全寫在同一個模組裡；但實務上比較常見的做法，是裝飾器定義在某一個模組，然後拿去套用在其他模組的函式身上，兩邊是分開的。第二個是，那支 `register` 收到什麼函式，就原封不動把同一個函式還回去；可是現實中大多數的裝飾器，都會在內部另外定義一個 `inner` 函式，再把那個 `inner` 回傳出去——也就是會掉包。

不過話說回來，就算 `register` 這種「原樣奉還、完全不動手腳」的寫法，也絕對不是沒路用的廢招。很多 Python 框架裡都看得到長得很像的裝飾器，它們的任務就是把函式登記進某一份「中央名冊」。最典型的情境，就是一份把網址規則對應到「負責產生 HTTP 回應的函式」的名冊。

這你可以想成公司總機那本分機表：外面的人打進某個分機號碼（對應到某個網址），總機就照著表把電話轉接給負責的那個人（對應到某個函式）。註冊型裝飾器幹的事，就是在每個函式「報到」的當下，順手把它的分機號碼登記進這本總機表裡頭。至於登記完之後，它要不要再對這個函式動點手腳、把人換掉，那就看狀況——可改可不改，兩種都有。

關於註冊型裝飾器實際套用的完整情況，後面有一堂專門講「用裝飾器強化的策略模式（Strategy Pattern）」的課會帶到，到時候再仔細看。

但你要記得，絕大多數的裝飾器，確實是會去改動被裝飾的函式的。而它們改動的標準手法，就是在裡頭定義一個 `inner` 函式，再把這個 `inner` 回傳出去、頂替掉原本那個函式。而只要程式碼用到了這種「函式裡再包一層 `inner` 函式」的寫法，它幾乎一定得靠閉包（closure）才能正確運作。

所以，想真正搞懂閉包，我們得先退一步，回頭把 Python 的「變數作用域（variable scope）是怎麼運作的」這件事重新弄清楚——這正是下一段要切進去的地方。

順帶補一個邏輯相同、但更貼近實戰的觀念：你之後在那些網頁框架裡會看到 `@route('/users')` 這種寫法，它其實就是這堂課「註冊型裝飾器」的進階版——一樣是把函式登記進中央名冊，只是多吃了一個參數（那串網址），好讓總機表知道該把哪個網址轉接給這個函式。骨子裡是同一套思路，等你把閉包跟帶參數的裝飾器都學完，回頭看這種寫法就會覺得理所當然了。

變數作用域規則（Variable Scope Rules）

這一段我們要定義並測試一個函式，它裡頭讀了兩個變數：一個是區域變數 `a`——它是用函式參數的形式定義的；另一個是 `b`，而 `b` 在函式裡頭哪裡都沒有被定義過。

```
>>> def f1(a):
...     print(a)
...     print(b)
...
>>> f1(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in f1
NameError: global name 'b' is not defined
```

這個錯誤一點都不意外：a 有（從參數傳進來的 3），但 b 從來沒人給過值，所以 Python 找不到 b，直接報 NameError。接著我們在外面幫全域的 b 賦個值，再呼叫一次，它就正常了：

```
>>> b = 6
>>> f1(3)
3
6
```

到這裡都很直覺。但接下來這個，可能會嚇你一跳。

看下面的 f2。它前兩行跟剛剛的 f1 一模一樣，差別只在最後多了一行「對 b 賦值」。照理說多賦一個值應該更安全才對，結果它反而在「第二個 print」就掛掉了——而且是在那行賦值還沒輪到執行之前就先掛了：

```
>>> b = 6
>>> def f2(a):
...     print(a)
...     print(b)
...     b = 9          # 就是這一行，惹出了大麻煩
...
>>> f2(3)
3
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in f2
UnboundLocalError: local variable 'b' referenced before assignment
```

注意輸出開頭是 3，證明 `print(a)` 確實跑過了。但第二個 `print(b)` 從頭到尾沒執行成功。第一次看到這個的人幾乎都會愣住：外面明明有個全域 `b = 6` 啊，而且那行 `b = 9` 還寫在 `print(b)` 後面，怎麼會印不出 6？

關鍵在這裡：當 Python 在「編譯函式本體」的時候，它一看到函式裡有 `b = 9` 這個賦值動作，就當場拍板認定「`b` 是區域變數」。這個判斷是在編譯期就定死的，跟程式碼寫的先後順序完全無關。編譯出來的位元碼（bytecode）就照這個決定走，之後執行時會堅持「去區域作用域裡撈 `b`」。所以當你呼叫 `f2(3)`，函式先抓到並印出區域變數 `a` 的值（3），但接著要去抓區域變數 `b` 的時候，發現這個 `b` 雖然「掛了名」卻還沒被賦值（unbound），於是當場報錯。

你可以把它想成公司報到的規矩：Python 在編譯時掃過函式一遍，只要看到某個名字在函式裡「有被賦值」，就先幫它辦好一張「本部門（local）員工證」，哪怕那行賦值寫在後面也一樣——身分在報到那刻就釘死了，不會因為程式碼順序而改變。所以當 `print(b)` 想叫 `b` 出來，警衛（直譯器）翻本部門名冊一看：這個人證件是辦了，但本人還沒到職，只好把你擋在門口。

這不是 bug，是 Python 刻意的設計選擇：Python 不強迫你宣告變數，但它預設「只要在函式本體裡被賦過值的變數，就是區域變數」。比起 JavaScript 這點其實安全多了——JS 同樣不強迫宣告，可是萬一你忘了用 `var` 把某個變數宣告成區域的，你很可能在完全沒察覺的情況下，就把外面的全域變數給蓋掉了。Python 這套預設規則，反而幫你擋掉了這種暗坑。

那如果我就是想讓直譯器把 `b` 當成全域變數來看待，而且還要在函式裡改它的值，該怎麼辦？用 `global` 宣告就行：

```
>>> b = 6
>>> def f3(a):
...     global b           # 明白告訴 Python：這個 b 是外面那個全域的，別當成 local
...     print(a)
...     print(b)
...     b = 9             # 這行改的，就是外面那個全域 b
...
>>> f3(3)
3
```

```
6
>>> b                # 函式跑完後再看外面的 b
9                    # 它真的被改成 9 了
```

從前面這幾個情況裡，我們其實已經看到兩種作用域在運作了。一種是「模組全域作用域 (module global scope)」，它由那些「定義在任何類別或函式區塊之外」的名字所組成；另一種是「函式區域作用域」，像 f3 裡頭的那些名字，來源不外乎是函式的參數、或是直接在函式本體裡被賦值的變數。除了這兩種，其實還有第三種變數來源，我們把它叫做 `nonlocal`，它正是閉包的核心關鍵——這個我們下一段就會碰到。

把變數作用域的運作看清楚之後，接下來就可以正式進攻閉包了，那是下一堂「閉包 (Closures)」的課要講的主軸。

如果你對 f1 跟 f2 在位元碼層級到底差在哪裡感到好奇，那就要靠 `dis` 這個模組了。它能很方便地把 Python 函式反組譯 (disassemble) 出來，讓你直接看到底層的指令：

```
>>> from dis import dis
>>> dis(f1)
```

f1 反組譯出來，你會看到它在處理 b 的時候，用的指令是 **LOAD_GLOBAL**——意思是「去全域作用域把 b 撈出來」。這跟我們的認知一致：f1 裡沒對 b 賦值，所以編譯器把它當全域。

但對照 f2 的反組譯：

```
>>> dis(f2)
```

你會發現，f2 處理同一個 b 的時候，用的指令卻變成了 **LOAD_FAST**——這是「去區域作用域快速取值」的指令。這就鐵證如山地說明了：編譯器把 f2 裡的 b 當成區域變數，而且就算那行 `b = 9` 的賦值是寫在後面才出現的也照算不誤，因為「一個變數到底是不是區域的」這件事，在整個函式本體裡是不會中途改變的——編譯期定了就是定了。

最後補個背景知識：真正在跑這些位元碼的 CPython 虛擬機 (VM) 是一台「堆疊機 (stack machine)」，所以你會看到 **LOAD** (載入)、**POP** (彈出) 這類字眼，指的都是對那個堆疊的操作。Python 各個操作碼 (opcode) 細部的意思，要再展開講就超出這門課的範圍了，但

它們全都跟 `dis` 模組一起，記載在官方文件「`dis——Python 位元碼的反組譯器`」那一節裡，有興趣可以自己翻去看。

閉包 (Closures)

閉包這個概念，幾乎只在「函式裡又包了函式」（巢狀函式，`nested functions`）的時候才會派上用場；反過來說，也只有在你有巢狀函式時，閉包才有意義。正因為這兩件事綁在一起，很多人都是同時把它們一起學起來的。

講得精確一點：閉包就是一個函式——我們姑且叫它 `f`——它的作用域被「延伸」了，延伸到把「`f` 本體裡有用到、但既不是全域變數、也不是 `f` 自己的區域變數」的那些變數也涵蓋進來。而這種變數，一定是來自「某個把 `f` 包在裡面的外層函式」的區域作用域。這個函式是不是匿名的（`anonymous`）其實不重要，重點只在於：它有辦法去存取那些「定義在它身體外面、卻又不是全域」的變數。

這觀念光用講的很抽象，直接拿個情境來看會清楚很多。假設我們要做一個 `avg` 函式，專門用來算「一串會不斷變長的數值」的平均——比方說，某項商品從上市到現在、整段歷史所有收盤價的平均。每天都會多進來一個新價格，而平均要把「到目前為止的所有價格」全部算進去。

從一張白紙開始，`avg` 用起來會像這樣：

```
>>> avg(10)
10.0
>>> avg(11)
10.5
>>> avg(12)
11.0
```

問題來了：這個 `avg` 到底是哪來的？而它又是把「過去那些值的歷史紀錄」藏在哪裡？

先看一個用類別寫的版本：

```
class Averager:
    def __init__(self):
        self.series = []           # 用實例屬性 series 來存歷史紀錄
```

<code>def __call__(self, new_value):</code>	<code># 定義了 __call__，實例就能像函式一樣被呼叫</code>
<code>self.series.append(new_value)</code>	<code># 把新值塞進帳本</code>
<code>total = sum(self.series)</code>	<code># 把帳本裡所有值加總</code>
<code>return total / len(self.series)</code>	<code># 回傳目前的平均</code>

Averager 做出來的實例是「可呼叫的」：

```
>>> avg = Averager()
>>> avg(10)
10.0
>>> avg(11)
10.5
>>> avg(12)
11.0
```

接著看一個用函數式寫法的版本，靠的是一個「高階函式」 `make_averager`：

<code>def make_averager():</code>	
<code>series = []</code>	<code># series 在 make_averager 本體裡被賦值，所以它是 make_averager 的區域變數</code>
 <code>def averager(new_value):</code>	<code># 在裡頭又包了一個 averager 函式</code>
<code>series.append(new_value)</code>	<code># 注意：這裡用到的 series，是外層那個 series</code>
<code>total = sum(series)</code>	
<code>return total / len(series)</code>	
 <code>return averager</code>	<code># 回傳 averager 這個函式物件本身（沒加括號）</code>

當你呼叫 `make_averager`，它會還你一個 `averager` 函式物件。之後每次呼叫這個 `averager`，它就把傳進來的引數塞進 `series`、再算出當下的平均：

```
>>> avg = make_averager()
>>> avg(10)
10.0
>>> avg(11)
10.5
>>> avg(12)
11.0
```

兩個版本擺在一起，相似度高得驚人：我們都是先呼叫 `Averager()` 或 `make_averager()` 拿到一個可呼叫物件 `avg`，接著只要 `avg(n)` 就能把 `n` 納進歷史、拿到更新後的平均。前者的 `avg` 是 `Averager` 的一個實例，後者的 `avg` 則是那個內層函式 `averager`。

差別在「歷史紀錄藏在哪」。類別版很好懂——就放在 `self.series` 這個實例屬性裡，清清楚楚。但函式版那個 `avg`，它到底是去哪裡找到 `series` 的？

這就是整段最關鍵、也最違反直覺的地方。`series` 明明是 `make_averager` 的區域變數（因為 `series = []` 寫在它的本體裡）。可是當你呼叫 `avg(10)` 的時候，`make_averager` 老早就執行完、回傳結束了，照理講它的區域作用域應該早就煙消雲散——但 `series` 居然還活著、還記得之前所有的值。

關鍵就在這：在 `averager` 的眼裡，`series` 是一個「自由變數（free variable）」。這是個專有名詞，意思是「一個沒有被綁定在區域作用域裡的變數」。外層的 `series = []` 屬於閉包涵蓋的範圍，而內層 `averager` 裡用到的 `series` 就是自由變數，兩個 `series` 之間有一條線串著——表示 `averager` 的作用域被「往外延伸」了，延伸到把外層那個 `series` 的綁定也整個收進自己懷裡。

打個比方就秒懂了：`make_averager` 像一間會計事務所，它每次開張就準備一本全新的空白帳本（`series = []`），然後派一個會計（`averager`）出去辦事。事務所派完人就關門了（函式 `return` 結束），可是這個會計手上死死抱著那本帳本——他走到哪帶到哪，帳本不會因為事務所關門就跟著不見。你每丟一個數字給他，他就記進帳本、當場算平均給你看。那本帳本對會計來說就是自由變數：不是他自己生的（不是他的 `local`），也不是全世界公用的（不是 `global`），而是「他出生的那間事務所留給他、讓他一輩子帶著走」的東西。

這種「函式記得自己出生環境變數」的本事，Python 是怎麼辦到的？祕密藏在函式物件的兩個特殊屬性裡。我們可以直接掀開來看：

```
>>> avg.__code__.co_varnames          # 這個函式真正屬於自己的區域名字
('new_value', 'total')
```

```
>>> avg.__code__.co_freevars          # 這個函式用到的自由變數名字
('series',)
```

`__code__` 代表的是函式被編譯後的本體。其中 `co_varnames` 列出的是 `averager` 自己的區域變數（收進來的參數 `new_value`、跟函式裡賦值的 `total`），而 `co_freevars` 列出的就是那些「外借的」自由變數——果然 `series` 在這裡。

那 `series` 的「值」本身，則被收在回傳函式 `avg` 的 `__closure__` 屬性裡。`avg.__closure__` 裡的每一項，都一一對應到 `co_freevars` 裡的一個名字。這些項目叫做「cell（細胞 / 儲存格）」，而每個 cell 身上有個 `cell_contents` 屬性，裡頭裝的才是真正的值：

```
>>> avg.__code__.co_freevars
('series',)
>>> avg.__closure__
(<cell at 0x107a44f78: list object at 0x107a91a48>,)
>>> avg.__closure__[0].cell_contents
[10, 11, 12]
```

看到沒——`cell_contents` 一打開，裡面正是 `[10, 11, 12]`，也就是我們前面餵進去的那三個值，原原本本地被那個會計抱在懷裡。

所以一句話總結閉包：閉包就是一個函式，它把「自己被定義當下存在的那些自由變數的綁定」整個保留下來，好讓函式日後被呼叫時，即使當初定義它的那個作用域早就不在了，它照樣用得到那些變數。

最後釐清一個邊界：一個函式會需要去處理「外部的、又不是全域的變數」，這種情況其實只會發生在一種場景——它被巢狀包在另一個函式裡，而那些變數正好是外層函式區域作用域的一部分。換句話說，閉包不會憑空出現，它一定是「函式包函式」這種結構底下的產物。

順帶接一句：你之後在學裝飾器時會發現，前面那堂課講的「裝飾器通常會定義一個 `inner` 函式再回傳出去」，骨子裡靠的就是這套閉包機制——`inner` 之所以能記得外層傳進來的東西，正是因為那些東西對它來說是抱在懷裡的自由變數。閉包搞懂了，裝飾器後半段就通了一大半。

nonlocal 宣告 (The nonlocal Declaration)

我們之前那版 `make_averager` 其實效率不太好。前一堂課的做法是把所有歷史值通通存進 `series` 這串列，然後每次 `averager` 被呼叫時，都重新把整串加總一遍。比較聰明的做法，是只存「累計總和」跟「目前累計了幾筆」這兩個數字就好，平均直接拿這兩個數字一除就出來，根本不必留著全部歷史。

下面這版就是照這個想法寫的，但它是「壞掉的」——故意擺給你看，你能看出它斷在哪裡嗎？

```
def make_averager():
    count = 0                # 累計幾筆
    total = 0                # 累計總和

    def averager(new_value):
        count += 1           # 想把 count 加 1
        total += new_value   # 想把 total 累加
        return total / count

    return averager
```

實際拿來跑，你會撞到這個：

```
>>> avg = make_averager()
>>> avg(10)
Traceback (most recent call last):
...
UnboundLocalError: local variable 'count' referenced before assignment
```

問題出在 `count += 1` 這行。當 `count` 是數字、或任何不可變（immutable）型別時，`count += 1` 跟 `count = count + 1` 是完全同一件事——也就是說，我們其實是在 `averager` 的本體裡「對 `count` 做了賦值」。還記得前面那堂作用域的課嗎？只要在函式本體裡對某個名字賦值，Python 就會在編譯期一口咬定它是這個函式的「區域變數」。於是 `count` 被當成 `averager` 自己的 `local`，可是它在 `count += 1` 的右邊就要先被讀取，這時它還沒被賦值，就爆了 `UnboundLocalError`。`total` 也是一模一樣的下場。

奇怪的是，之前用 `series` 的版本怎麼就沒這個毛病？關鍵在於：我們從頭到尾「沒有重新賦值給 `series`」這個名字，我們只是呼叫 `series.append(...)`、再對它 `sum`、`len`——這些都是「在原地改它的內容」，不是「換一個新的 `series` 給它」。所以 `series` 始終是同一個物件、始終保持自由變數的身分。我們等於是占了「串列是可變（mutable）」的便宜。

但碰到數字、字串、tuple 這類不可變型別，你就只能「讀」，沒辦法「原地更新」。你唯一能做的就是整個換掉它（rebind），像 `count = count + 1` 這樣。而你一這麼寫，等於在偷偷宣告「我要在這個函式裡新生一個叫 count 的變數」——它就不再是外借的自由變數了，自然也就不會被存進閉包裡。

打個比方：串列像一本「活頁帳本」，你一直往裡頭加頁、塗改內容（append），但帳本本身永遠是同一本，所以會計手上抓著的還是它，沒問題。可是數字像一塊「刻死的牌子」，你沒辦法改牌子上的字，只能整塊丟掉、換一塊新的——而你一換新牌子，Python 就誤以為你要在內層自立門戶、開一個自己的 count，麻煩就來了。

為了繞過這個困境，Python 3 引進了 nonlocal 這個關鍵字。它讓你能夠「即使在函式裡對某個變數賦值，也照樣把它宣告成自由變數」。而且，當你對一個 nonlocal 變數賦新值時，連帶閉包裡存的那份綁定也會跟著被改掉。它就像一張正式的委任狀：你在內層函式開頭蓋章聲明「這幾個名字不是我內層自己生的，它們是外層的，我要直接動外層那一份」。

修好的正確版本長這樣：

```
def make_averager():
    count = 0
    total = 0

    def averager(new_value):
        nonlocal count, total          # 委任狀：count 和 total 都是外層的，別當成我的 local
        count += 1
        total += new_value
        return total / count

    return averager
```

加上這一行 `nonlocal count, total` 之後，Python 就不會再幫你另生 local，而是乖乖去改外層那份綁定，閉包裡的值也同步更新——一切就順了。

順帶提一個同邏輯下的替代做法：在 nonlocal 還沒問世的年代，有人會「不換牌子、改用可變容器」來繞過——比方把 count、total 塞進一個串列或一個小物件裡，然後永遠只改它的內容、絕不 rebind，這樣就不會觸發「被當成 local」的判定。骨子裡思路是一樣的，但 nonlocal 是 Python 3 給的正規解，語意最直白，看程式碼的人一眼就懂你的意圖，所以現在沒理由再用那種拐彎的招了。

變數查找邏輯 (Variable Lookup Logic)

把 `nonlocal` 搞懂之後，正好可以把 Python「到底怎麼決定一個變數從哪裡來」的整套邏輯收個尾。當一個函式被定義時，位元碼編譯器就會針對函式裡出現的某個變數 `x`，照下面這套順序決定該怎麼抓它。

最優先的是看有沒有明確宣告。如果函式裡寫了 `global x`，那 `x` 就直接認定為全域變數，讀取和賦值都對著全域那一份來。如果寫的是 `nonlocal x`，那 `x` 就來自「離它最近、且有定義 `x` 的那個外層函式」的區域變數，讀寫也都對著那一份。

沒有宣告的話，就看 `x` 在這個函式裡有沒有「被賦值」或「身為參數」。只要 `x` 是參數、或在函式本體裡被賦過值，它就是這個函式的區域變數，到此為止。

最後一種情況，是 `x` 在函式裡只被「讀取」、卻從沒被賦值、也不是參數。這時 Python 會像剝洋蔥一樣一層層往外找：先到外層那些函式本體的區域作用域（也就是 `nonlocal` 作用域）裡找；外層都找不到，就往模組的全域作用域去讀；連全域都沒有，最後才到內建命名空間 `__builtins__.__dict__` 裡去撈（像 `print`、`len`、`sum` 這些就是住在這層）。一路找到底還是沒有，才會真的報錯。

到這裡，Python 的閉包我們算是徹底攻下來了。手上有了閉包這把鑰匙，接下來就能真正動手，用巢狀函式扎扎实實地把裝飾器給實作出來了。

實作一個簡單的裝飾器 (Implementing a Simple Decorator)

把前面閉包那幾堂課的功力全部集合起來，我們終於可以動手做一個真正派得上用場的裝飾器了。這個裝飾器叫 `clock`，它的任務是幫被裝飾的函式「計時」：每次那個函式被呼叫，它就量出這趟跑了多久、帶了哪些引數進去、又算出了什麼結果，然後印出來。

```
import time

def clock(func):
    def clocked(*args):
        t0 = time.perf_counter()
        result = func(*args)
        elapsed = time.perf_counter() - t0
        name = func.__name__
        arg_str = ', '.join(repr(arg) for arg in args)
        print(f'[{elapsed:0.8f}s] {name}({arg_str}) -> {result!r}')
    # 內層 clocked 用 *args 接收「任意數量的位置引數」
    # 按下碼錶，記下起跑時間
    # 讓本尊 func 真正進去辦事，結果先收著
    # 按停碼錶，算出這趟耗時
    # 抓出本尊函式的名字
    # 把每個引數轉成可讀字串、用逗號串起來
    # 印出「耗時、名字、引數、結果」
```

```
    return result
    return clocked
```

```
# 把本尊的結果原封不動交還出去
# 回傳 clocked，拿它去頂替被裝飾的函式
```

你可以把 clock 想成「在每個函式門口安插一個碼錶登記員」。每次有人（呼叫）上門辦事，這個登記員先咻一聲按下碼錶，讓本尊進去把事辦完（`func(*args)`），本尊一出來就按停碼錶算耗時，接著把「花了多久、帶什麼進去、辦出什麼結果」全寫在板子上唸出來，最後再把結果原封不動遞還給上門的人——登記員只是在旁邊計時播報，完全不干涉本尊辦事的內容。

這裡有一行是整個機制的命脈：`result = func(*args)`。clocked 之所以叫得動 func，靠的就是閉包——func 對 clocked 來說是個自由變數，是外層 clock 在派它出門時塞給它、讓它一輩子抱著走的東西。前面幾堂課講的閉包、自由變數，在這裡全都派上用場了。最後 `return clocked` 這一步，則是把 clocked 回傳出去、頂替掉原本被裝飾的函式，也就是上上堂課講的「掉包」。

接著看怎麼用它：

```
import time
from clockdeco0 import clock

@clock
def snooze(seconds):
    time.sleep(seconds)

@clock
def factorial(n):
    return 1 if n < 2 else n * factorial(n-1)

if __name__ == '__main__':
    print('*' * 40, 'Calling snooze(.123)')
    snooze(.123)
    print('*' * 40, 'Calling factorial(6)')
    print('6! =', factorial(6))
```

跑起來大概長這樣（那串秒數每台機器、每次跑都會不一樣，看個感覺就好）：

```
$ python3 clockdeco_demo.py
***** Calling snooze(.123)
[0.12363791s] snooze(0.123) -> None
***** Calling factorial(6)
[0.00000095s] factorial(1) -> 1
[0.00002408s] factorial(2) -> 2
[0.00003934s] factorial(3) -> 6
[0.00005221s] factorial(4) -> 24
[0.00006390s] factorial(5) -> 120
[0.00008297s] factorial(6) -> 720
6! = 720
```

這裡有個很精妙、卻很容易被滑過去的點：我們只呼叫了一次 `factorial(6)`，為什麼輸出卻從 `factorial(1)` 一路印到 `factorial(6)`，每一層都被計時了？因為 `factorial` 被裝飾之後，這個名字早就指向 `clocked` 了，而 `factorial` 內部那行遞迴 `factorial(n-1)` 呼叫的也是這個名字——所以每往下遞迴一層，都會再經過一次碼錶登記員。本尊的遞迴邏輯沒變，但每一層都被順手計了時，這正是裝飾器「掉包」之後產生的連鎖效果。

它是怎麼運作的 (How It Works)

記住，下面這段：

```
@clock
def factorial(n):
    return 1 if n < 2 else n * factorial(n-1)
```

骨子裡做的就是這件事：

```
def factorial(n):
    return 1 if n < 2 else n * factorial(n-1)
```

```
factorial = clock(factorial)
```

所以兩種寫法裡，`clock` 拿到的 `func` 引數都是那個 `factorial` 函式。`clock` 接著做出並回傳 `clocked`，而 Python 直譯器就把這個 `clocked` 指派回 `factorial` 這個名字（用 `@` 的寫法是在幕後偷偷幫你做掉的）。不信的話，你把模組匯入進來，去查 `factorial` 的名字看看：

```
>>> import clockdeco_demo
>>> clockdeco_demo.factorial.__name__
'clocked'
```

果然，`factorial` 現在握的其實是 `clocked` 的參照。從這一刻起，你每次呼叫 `factorial(n)`，實際被執行的都是 `clocked(n)`。而 `clocked` 內部做的，說穿了就是一條流水線：先記下起跑時間 `t0`，接著呼叫原本的 `factorial` 並把結果收好，然後算出這趟花了多少時間，再把蒐集到的這些資料排版印出來，最後把剛剛收好的那個結果回傳出去。本尊負責算，分身負責計時播報，各司其職。

不過，`clock` 這版其實有兩個小缺陷。第一，它的 `clocked` 只寫了 `*args`，完全沒處理關鍵字引數（keyword arguments），所以你一旦用 `func(x, y=2)` 這種帶名字的引數呼叫就會出包。第二，掉包之後，被裝飾函式原本的名字（`__name__`）跟說明文件（`__doc__`）都被分身蓋掉了——這也是剛剛查到名字變成 `'clocked'` 的原因。

要把這兩個毛病一次治好，正規做法是搬出標準函式庫裡的 `functools.wraps`，讓它幫忙把本尊身上那些重要屬性複製到 `clocked` 上；順便把關鍵字引數也補上正確處理。改良版如下：

```
import time
import functools

def clock(func):
    @functools.wraps(func)
    def clocked(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - t0
        name = func.__name__

        # 幫分身 clocked 冒用本尊 func 的名字與說明文件
        # 這次連關鍵字引數 **kwargs 也一起收
        # 位置引數、關鍵字引數一起原樣轉交給本尊
```

```
arg_lst = [repr(arg) for arg in args]          # 先處理位置引數
arg_lst.extend(f'{k}={v!r}' for k, v in kwargs.items()) # 再把關鍵字引數補成 k=v 的形式
arg_str = ', '.join(arg_lst)
print(f'[{elapsed:0.8f}s] {name}({arg_str}) -> {result!r}')
return result
return clocked
```

那行 `@functools.wraps(func)` 就是前面那堂課我提過、現在正式登場的東西——它像幫分身辦一張「合法冒用本尊身分」的證件，貼上去之後，`clocked` 對外查起來名字還是原本的 `factorial`、說明文件也還是本尊那一份，計時功能照常，身分卻不會再被搞混。這就是所謂「行為端正的裝飾器」該有的樣子。

`functools.wraps` 只是標準函式庫裡眾多現成裝飾器的其中一個而已。下一段，我們要來會會 `functools` 提供的、戰力最猛的那一個裝飾器——`cache`。

標準函式庫裡的裝飾器 (Decorators in the Standard Library)

Python 內建了三個專門用來裝飾「方法」的東西：`property`、`classmethod`、`staticmethod`。其中 `property` 會在後面「用 `Property` 做屬性驗證」那堂課細講，另外兩個則留到「`classmethod` 對上 `staticmethod`」那堂課再談。上一段我們也已經見過另一個很重要的裝飾器——`functools.wraps`，它是幫你打造「行為端正的裝飾器」的好幫手。

不過標準函式庫裡最有意思的幾個裝飾器，要算是 `cache`、`lru_cache` 跟 `singledispatch` 了，它們全都出自 `functools` 模組。接下來就一個一個來會會它們。

用 `functools.cache` 做記憶化 (Memoization with `functools.cache`)

`functools.cache` 這個裝飾器實作的是「記憶化 (memoization)」：一種把「昂貴函式」過去算過的結果先記下來的最佳化手法，這樣同一組引數再進來時，就直接拿之前的答案，免得重算一遍。

你可以把它想成考試帶小抄：某一題你已經算過了，就把答案抄在小抄上；下次再碰到一模一樣的題目，直接照抄，完全不用再從頭演算。

最能凸顯它威力的，就是拿它來救那個「慢到令人髮指」的遞迴版費氏數列 (Fibonacci)：

```
from clockdeco import clock

@clock
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 2) + fibonacci(n - 1)

if __name__ == '__main__':
    print(fibonacci(6))
```

跑起來，除了最後一行的答案以外，其餘每一行都是 clock 裝飾器印出來的計時紀錄，而且你會看到滿坑滿谷的重複呼叫：

```
[0.00000042s] fibonacci(0) -> 0
[0.00000049s] fibonacci(1) -> 1
[0.00006115s] fibonacci(2) -> 1
[0.00000031s] fibonacci(1) -> 1
[0.00000935s] fibonacci(0) -> 0
[0.00000030s] fibonacci(1) -> 1
[0.00001084s] fibonacci(2) -> 1
[0.00002074s] fibonacci(3) -> 2
... (中間還有一大票一模一樣的呼叫被反覆計算) ...
[0.00006670s] fibonacci(5) -> 5
[0.00016852s] fibonacci(6) -> 8
8
```

浪費得很明顯：光是 `fibonacci(1)` 就被呼叫了八次，`fibonacci(2)` 被算了五次。因為遞迴的兩條岔路 `fibonacci(n-2)` 跟 `fibonacci(n-1)` 各自又會重新去算一堆同樣的小題目，完全沒有人記得「這題我剛剛才算過」。

但只要加兩行、把 cache 掛上去，效能立刻天差地遠：

```

import functools
from clockdeco import clock

@functools.cache          # 這行需要 Python 3.9 以上；舊版的替代寫法見下一段 lru_cache
@clock
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 2) + fibonacci(n - 1)

if __name__ == '__main__':
    print(fibonacci(6))

```

這裡要特別點出一個重點：`@functools.cache` 跟 `@clock` 是「疊起來」用的（stacked decorators）。疊法的意思是——`@cache` 是套在「`@clock` 處理完所回傳的那個函式」之上的。讀的時候由下往上看：先被 `clock` 包一層、再被 `cache` 包一層。

加了 `cache` 之後，`fibonacci` 對每個 `n` 都只會真正被呼叫「一次」：

```

[0.00000043s] fibonacci(0) -> 0
[0.00000054s] fibonacci(1) -> 1
[0.00006179s] fibonacci(2) -> 1
[0.00000070s] fibonacci(3) -> 2
[0.00007366s] fibonacci(4) -> 3
[0.00000057s] fibonacci(5) -> 5
[0.00084795s] fibonacci(6) -> 8
8

```

差距有多誇張？另一個測試裡，要算 `fibonacci(30)`：有 `cache` 的版本只用了 31 次必要的呼叫，總共花 0.00017 秒就跑完；而沒 `cache` 的版本在一台 Intel Core i7 筆電上，整整跑了 12.09 秒——因為它光是 `fibonacci(1)` 就呼叫了 832,040 次，全部加起來高達 2,692,537 次呼叫。同一件事，一個眨眼，一個泡杯咖啡。

不過 `cache` 有個前提：被裝飾函式收的所有引數都必須是「可雜湊的 (hashable)」。原因是它底層靠的 `lru_cache` 是用一個字典 (dict) 來存結果的，而那本字典的鍵 (key)，正是由呼叫時用到的位置引數跟關鍵字引數組出來的——當鍵就得能雜湊，所以像 `list` 這種不可雜湊的引數就不能用。

使用 `lru_cache` (Using `lru_cache`)

其實 `functools.cache` 只是個薄薄的外殼，包在更老資格的 `functools.lru_cache` 外面而已。`lru_cache` 比較有彈性，而且能相容到 Python 3.8 甚至更早的版本。

`lru_cache` 最大的優點，是它的記憶體用量會被 `maxsize` 這個參數「框住上限」。`maxsize` 預設值定得相當保守，是 128——意思是這本小抄無論何時最多就只記 128 筆。LRU 是「Least Recently Used (最近最少使用)」的縮寫，意思是：當位子滿了，那些「很久沒被翻過」的舊紀錄就會被丟掉，空出位置給新的——就像小抄寫滿了，你會把最久沒用到的那幾題擦掉，挪空間給新題目。

從 Python 3.8 開始，`lru_cache` 有兩種用法。最簡單的是直接掛：

```
@lru_cache
def costly_function(a, b):
    ...
```

另一種（從 3.2 就有）是把它當函式呼叫、加上一對括號：

```
@lru_cache()
def costly_function(a, b):
    ...
```

這兩種寫法用的都是預設參數，也就是 `maxsize=128` 跟 `typed=False`。

`maxsize` 設定的是最多存幾筆，小抄寫滿後，每要塞一筆新的，就丟掉一筆最久沒用的。為了效能最佳，`maxsize` 最好設成 2 的次方。如果你傳 `maxsize=None`，等於把 LRU 那套淘汰邏輯整個關掉，小抄跑得更快、但永遠不丟東西，代價是記憶體可能會吃到撐爆——而這，其實就正是 `@functools.cache` 在做的事（難怪它是 `cache` 的本體）。

至於 `typed`，管的是「不同型別的引數要不要分開存」。預設 `typed=False` 時，被視為相等的浮點數跟整數只會存成同一筆，所以呼叫 `f(1)` 跟 `f(1.0)` 會共用一個位子。如果設成 `typed=True`，這兩種就會被當成不同的引數、各存一筆，有可能存下不一樣的結果。帶非預設參數的寫法像這樣：

```
@lru_cache(maxsize=2*20, typed=True)
def costly_function(a, b):
    ...
```

單一分派泛型函式 (Single Dispatch Generic Functions)

接著來看另一個很強的裝飾器：`functools.singledispatch`。

想像我們要做一個「網頁應用程式的除錯工具」，需要幫各種不同型別的 Python 物件產生對應的 HTML 顯示。一開始可能會先寫出這樣一個函式：

```
import html

def htmlize(obj):
    content = html.escape(repr(obj))    # 把物件轉成字串、再做 HTML 跳脫，避免特殊字元出包
    return f'<pre>{content}</pre>'
```

這對任何型別都能動，但接下來我們想針對某些型別「客製化」它的顯示方式。比方說：碰到字串 (`str`)，就把裡頭的換行字元換成 `<br/>\n`、外面也改用 `<p>` 而不是 `<pre>`；碰到整數 (`int`)，要同時秀出十進位跟十六進位（而且布林值 `bool` 要特別處理）；碰到串列 (`list`)，要產出一份 HTML 清單，而且每一項還要依它自己的型別來格式化；碰到 `float` 跟 `Decimal`，除了照常顯示數值，順手也用分數的形式秀出來（為什麼不呢）。

我們想要的行為大概是這樣：

```
>>> htmlize({1, 2, 3})
'<pre>{1, 2, 3}</pre>'
```

```

>>> htmlize(abs)
'<pre>&lt;built-in function abs&gt;</pre>'
>>> htmlize('Heimlich & Co.\n- a game')
'<p>Heimlich & Co.<br/>\n- a game</p>'
>>> htmlize(42)
'<pre>42 (0x2a)</pre>'
>>> print(htmlize(['alpha', 66, {3, 2, 1}]))
<ul>
<li><p>alpha</p></li>
<li><pre>66 (0x42)</pre></li>
<li><pre>{1, 2, 3}</pre></li>
</ul>
>>> htmlize(True)
'<pre>True</pre>'
>>> htmlize(fractions.Fraction(2, 3))
'<pre>2/3</pre>'
>>> htmlize(2/3)
'<pre>0.6666666666666666 (2/3)</pre>'
>>> htmlize(decimal.Decimal('0.02380952'))
'<pre>0.02380952 (1/42)</pre>'

```

問題來了：Python 沒有 Java 那種「方法多載 (method overloading)」，所以我們沒辦法單純地針對每種型別，各寫一個簽名不同、但同名的 `htmlize`。一種土法煉鋼的解法，是把 `htmlize` 變成一個「分派函式」，裡頭擺一長串 `if/elif` 或 `match/case`，分別去呼叫 `htmlize_str`、`htmlize_int` 這類各自的專責函式。但這做法有兩個大毛病：一是用你模組的人沒辦法自己擴充它；二是隨著時間過去，這個分派器會越長越肥，而它跟那些專責函式之間又綁得死死的，牽一髮動全身。

`functools singledispatch` 就是來解這個困境的。它讓「不同的模組」都能各自貢獻一塊解法，甚至連那些屬於第三方套件、你根本改不到的型別，你也能輕鬆替它們補上專責函式。

你可以把它想成公司大廳的接待總櫃台：訪客一進門，總機看「來訪者的身分（也就是第一個引數的型別）」，自動把他轉接給對應的專員——字串訪客轉給字串專員、整數訪客轉給整數專員。而且每個專員是自己跑去櫃台「登記」自己負責哪一類訪客的，總機完全不必寫死一長串「如果是 A 就...如果是 B 就...」。這正是「單一分派 (single dispatch)」的意思：只看「第一個引數」的型別來決定派工。如果是看好幾個引數來決定，那就叫「多重分派 (multiple dispatch)」了。

只要你用 `@singledispatch` 去裝飾一個普通函式，它就搖身一變成為一個「泛型函式 (generic function)」的入口——所謂泛型函式，就是一群「做同一件事、但依第一個引數型別而有不同做法」的函式集合。實際長這樣：

```
from functools import singledispatch
from collections import abc
import fractions
import decimal
import html
import numbers

@singledispatch                                     # 標記出處理 object 型別的「基底函式」，當作全部落空時的總接待
def htmlize(obj: object) -> str:
    content = html.escape(repr(obj))
    return f'<pre>{content}</pre>'

@htmlize.register                                   # 每個專責函式都用 @基底.register 來登記
def _(text: str) -> str:                             # 第一個參數的型別註記 (str) 決定它何時被選用；函式名字不重要，用 _ 最乾脆
    content = html.escape(text).replace('\n', '<br/>\n')
    return f'<p>{content}</p>'

@htmlize.register                                   # 登記給「序列」這個抽象類別，而不是死板的 list
def _(seq: abc.Sequence) -> str:                     # 每一項再各自遞迴呼叫 htmlize，依它自己的型別處理
    inner = '</li>\n<li>'.join(htmlize(item) for item in seq)
    return f'<ul>\n<li>' + inner + f'</li>\n</ul>'

@htmlize.register
def _(n: numbers.Integral) -> str:                   # 登記給「整數」這個抽象類別
    return f'<pre>{n} (0x{n:x})</pre>'                # 同時印十進位與十六進位

@htmlize.register
def _(n: bool) -> str:                               # bool 雖然是整數的子型別，仍給它專屬待遇
    return f'<pre>{n}</pre>'
```

```

@htmlize.register(fractions.Fraction)                # 不想加型別註記時，也可以把型別直接傳給 register
def _(x) -> str:
    frac = fractions.Fraction(x)
    return f'<pre>{frac.numerator}/{frac.denominator}</pre>'

@htmlize.register(decimal.Decimal)                  # register 會回傳「沒被裝飾的原函式」，所以能像這樣疊起來
@htmlize.register(float)                            # 一次把 Decimal 和 float 兩種型別登記到同一個實作上
def _(x) -> str:
    frac = fractions.Fraction(x).limit_denominator()
    return f'<pre>{x} ({frac.numerator}/{frac.denominator})</pre>'

```

這段裡有幾個眉角值得單獨拎出來講。首先，`@singledispatch` 標記的是那個處理 object 的基底函式，它扮演「總接待」——當訪客的型別跟其他所有專員都對不上時，就由它來收尾。其次，每個專員函式的名字其實一點都不重要，所以統一用底線 `_` 來命名，正好直白地宣告「這名字你別管，真正決定我何時上場的是參數的型別註記」。要新增一種型別的特殊待遇，就再登記一個新函式、在第一個參數標上對應的型別註記即可。

再來一個容易卡住的點：`bool` 明明是 `numbers.Integral` 的子型別，那 `htmlize(True)` 怎麼不會跑去整數那個實作？因為 `singledispatch` 的邏輯是「找型別吻合度『最精準』的那個實作」，而且跟它們在程式碼裡出現的先後順序完全無關。`bool` 比 `Integral` 更精準，所以 `True` 會準確地落到 `bool` 專員手上。另外，如果你不想、或沒辦法在函式上加型別註記，也可以像處理 `Fraction`、`Decimal`、`float` 那樣，直接把型別當引數傳進 `@基底.register(型別)`——這個語法從 Python 3.4 起就能用。而 `register` 這個裝飾器會把「沒被裝飾過的原函式」原樣回傳，正因如此你才能把好幾個 `register` 疊起來，一口氣把兩個以上的型別綁到同一個實作上（就像 `Decimal` 和 `float` 共用那段）。

這裡還藏著一個很值得內化的設計準則：能登記給「抽象基底類別（ABC）」就別登記給「具體型別」。也就是說，優先登記 `numbers.Integral`、`abc.Sequence` 這種抽象類別，而不是 `int`、`list` 這種具體實作。這樣你的程式碼就能支援更廣的相容型別——比方說，某個 Python 擴充套件提供了「固定位元長度的整數」當作 `numbers.Integral` 的子類別，你的程式碼不必改一個字就自動吃下它。

`singledispatch` 還有一個很漂亮的特質：你可以在系統的「任何地方、任何模組」裡登記專員函式。日後你新增一個模組、定義了一個新的自訂型別，只要順手替它登記一個專員就好；甚至連那些「不是你寫的、你也改不動的別人家的類別」，你照樣能替它們補上專屬的處理函式。整套機制鬆耦合又好擴充，是標準函式庫裡設計得相當周到的一員。它的功能比這裡講到的還多，想深入的話，PEP 443「單一分派泛型函式」是不錯的參考文件，只是它沒提到型別註記（那是後來才加進去的）；`functools` 模組的官方文件如今寫得更完整、也更新到位，在 `singledispatch` 那一條底下還附了好幾段說明，值得一看。

到目前為止，我們已經見過會「吃引數」的裝飾器了，像 `@lru_cache()` 就是。而前面那個用 `singledispatch` 拼出來的 `htmlize`，則展示了一個入口函式去調度一整組實作的巧妙手法。順著這條線，接下來要正式進攻的，就是「如何自己動手做一個帶參數的裝飾器」。

帶參數的裝飾器（Parameterized Decorators）

到這裡，我們要攻下函式裝飾器的最後一塊，也是最燒腦的一塊：讓裝飾器能「吃參數」。

先想清楚一件事。Python 在解讀一個裝飾器時，它的動作很死板——就是把「被裝飾的那個函式」當成「第一個引數」，塞給裝飾器函式。那問題來了：如果裝飾器函式被那個函式占走了第一個引數的位子，我要怎麼再額外傳別的參數給它？

答案是：做一間「裝飾器工廠」。這間工廠專門負責收下你那些額外的參數，然後現做、回傳一個真正的裝飾器，接著這個裝飾器才被套到待裝飾的函式上。聽起來繞口？確實。我們從見過最單純的那個裝飾器 `register` 切進去，一步步把這層繞口的東西拆開。先把簡化版的 `register` 擺在這裡方便對照：

```
registry = []

def register(func):
    print(f'running register({func})')
    registry.append(func)
    return func

@register
def f1():
    print('running f1()')
```

這個 `register` 一層搞定，沒有參數。現在我們要幫它加一個開關。

帶參數的註冊裝飾器（A Parameterized Registration Decorator）

我們想讓「要不要登記某個函式」這件事可以開關自如，所以替 `register` 加一個可選的 `active` 參數：當 `active` 是 `False` 時，就跳過登記、不把這個函式收進名冊（甚至把它從名冊移掉）。

觀念上，加了參數之後的 `register` 嚴格講已經「不是裝飾器」了，它升格成了「裝飾器工廠」。你呼叫它，它回傳的才是那個真正會被套到目標函式上的裝飾器：

```
registry = set()                                # 改用 set，新增和移除函式都更快

def register(active=True):                      # register 現在收一個可選的關鍵字引數
    def decorate(func):                         # decorate 才是「真正的裝飾器」，注意它才是收函式的那個
        print('running register'
              f'(active={active}) -> decorate({func})')
        if active:                             # 只有當 active (從閉包取得) 為 True 才登記
            registry.add(func)
        else:                                  # 若 active 為 False，且 func 已在名冊裡，就把它移除
            registry.discard(func)
        return func
    return decorate

@register(active=False)                         # 工廠必須「以函式形式呼叫」，帶上想要的參數
def f1():
    print('running f1()')

@register()                                     # 就算不帶參數，也得寫成 register()，才能拿到真正的裝飾器
def f2():
    print('running f2()')

def f3():
    print('running f3()')
```

這裡的層次是整段的關鍵，你可以這樣想：以前的 `register` 就是一個登記員，你直接把函式交給他。現在的 `register` 變成一間「客製化派遣公司」，你得先走進去告訴它「這次登記的開關要開還是要關」(`active`)，公司收下你的要求後，才現做一個對應的登記員 (`decorate`) 派出來，這個登記員再去處理你的函式。`decorate` 裡頭用到的 `active`，正是從外層 `register` 那裡抱過來的自由變數——又是閉包在撐場。

也因為多了這層工廠，有個很容易踩雷的地方：`@register()` 那對括號絕對不能省。哪怕你完全不想傳參數，也得寫 `@register()` 而不是 `@register`。因為 `register` 現在回傳的是裝飾器、不是被裝飾的函式；你得先呼叫它一次、讓它把 `decorate` 吐出來，`@` 才有東西可套。少了那對括號，套上去的就會是工廠本身，整個就錯了。

把這支放進 `registration_param.py`，匯入看看：

```
>>> import registration_param
running register(active=False) -> decorate(<function f1 at 0x10063c1e0>)
running register(active=True) -> decorate(<function f2 at 0x10063c268>)
>>> registration_param.registry
{<function f2 at 0x10063c268>}
```

注意名冊裡只有 `f2`，`f1` 沒進去——因為 `f1` 那邊傳的是 `active=False`，所以套到 `f1` 身上的那個 `decorate` 根本沒把它加進名冊。

如果不用 `@` 這個寫法，改把 `register` 當普通函式來用，那要裝飾函式 `f` 的完整寫法就是 `register()(f)`（把 `f` 加進名冊），或 `register(active=False)(f)`（不加、或移除）。下面這段把「動態加入與移除」演了一遍：

```
>>> from registration_param import *
running register(active=False) -> decorate(<function f1 at 0x10073c1e0>)
running register(active=True) -> decorate(<function f2 at 0x10073c268>)
>>> registry                                     # 一匯入，名冊裡就有 f2
{<function f2 at 0x10073c268>}
>>> register()(f3)                               # register() 回傳 decorate，再套到 f3 上
running register(active=True) -> decorate(<function f3 at 0x10073c158>)
<function f3 at 0x10073c158>
>>> registry                                     # 上一行把 f3 加進名冊了
{<function f3 at 0x10073c158>, <function f2 at 0x10073c268>}
>>> register(active=False)(f2)                   # 這次把 f2 從名冊移除
running register(active=False) -> decorate(<function f2 at 0x10073c268>)
<function f2 at 0x10073c268>
```

```
>>> registry                                     # 確認名冊裡只剩 f3
{<function f3 at 0x10073c158>}
```

看出那個 `register()(f3)` 的雙重括號了嗎？第一對括號呼叫工廠拿到 `decorate`，第二對括號才把 `decorate` 套到 `f3` 上。這正是 `@` 語法在幕後幫你做的兩件事。

帶參數的 clock 裝飾器 (The Parameterized Clock Decorator)

把同一套「工廠→裝飾器→包裝」的三層結構套回前面那個計時的 `clock`，我們就能加上一個很實用的功能：讓使用者自己傳一個「格式字串」進來，控制計時報告長什麼樣子。

```
import time

DEFAULT_FMT = '[{elapsed:0.8f}s] {name}({args}) -> {result}'

def clock(fmt=DEFAULT_FMT):
    # clock 是帶參數的裝飾器工廠
    def decorate(func):
        # decorate 才是真正的裝飾器
        def clocked(*_args):
            # clocked 包住被裝飾的函式
            t0 = time.perf_counter()
            _result = func(*_args)          # _result: 被裝飾函式真正的回傳值
            elapsed = time.perf_counter() - t0
            name = func.__name__
            args = ', '.join(repr(arg) for arg in _args)  # _args 是真引數，args 是給顯示用的字串
            result = repr(_result)          # result: _result 的字串表示，給顯示用
            print(fmt.format(**locals()))    # 關鍵：用 **locals() 把上面這些區域變數全攤平餵給 fmt
            return _result                  # clocked 頂替原函式，所以要把原函式的回傳值還回去
        return clocked
    return decorate

if __name__ == '__main__':
    @clock()
    # 自我測試：clock() 不帶引數，套用的裝飾器會用預設格式
    def snooze(seconds):
        time.sleep(seconds)
```

```
for i in range(3):
    snooze(.123)
```

這裡最值得單獨講的眉角，是 `print(fmt.format(**locals()))` 那一行。`locals()` 會回傳「clocked 當下所有區域變數」組成的字典，像 `elapsed`、`name`、`args`、`result` 都在裡頭；`**` 再把這本字典攤平成一個個關鍵字引數，送進 `fmt.format(...)`。這麼一搞的妙處在於：使用者的格式字串裡只要寫得出 `{elapsed}`、`{name}`、`{args}`、`{result}` 這些名字，就能任意挑選、任意排版——格式字串引用得到 `clocked` 裡的任何一個區域變數。彈性全靠這招撐起來。

直接跑，用的是預設格式：

```
$ python3 clockdeco_param.py
[0.12412500s] snooze(0.123) -> None
[0.12411904s] snooze(0.123) -> None
[0.12410498s] snooze(0.123) -> None
```

接著換個格式字串試試。傳一個只想看「名字加耗時」的格式：

```
import time
from clockdeco_param import clock

@clock('{name}: {elapsed}s')
def snooze(seconds):
    time.sleep(seconds)

for i in range(3):
    snooze(.123)
```

輸出就變成：

```
$ python3 clockdeco_param_demo1.py
snooze: 0.12414693832397461s
snooze: 0.1241159439086914s
snooze: 0.12412118911743164s
```

再換一個帶引數、又把耗時限制成小數三位的格式：

```
import time
from clockdeco_param import clock

@clock('{name}({args}) dt={elapsed:0.3f}s')
def snooze(seconds):
    time.sleep(seconds)

for i in range(3):
    snooze(.123)
```

輸出：

```
$ python3 clockdeco_param_demo2.py
snooze(0.123) dt=0.124s
snooze(0.123) dt=0.124s
snooze(0.123) dt=0.124s
```

同一支 clock，光靠那串格式字串，就變出三種完全不同的報告長相——這就是把參數收進工廠那一層所換來的彈性。

用類別實作的 clock 裝飾器 (A Class-Based Clock Decorator)

最後給你看一個邏輯完全相同、但風骨不一樣的寫法：把這個帶參數的 clock，改用「一個帶 `__call__` 的類別」來實作。這是有些人（像 Graham Dumpleton、Lennart Regebro 那一派）偏好的風格，你可以擺在一起比比看，自己喜歡哪一種。

```

import time

DEFAULT_FMT = '[{elapsed:0.8f}s] {name}({args}) -> {result}'

class clock:
    # 不再用外層 clock 函式，改用 clock 類別當「工廠」
    def __init__(self, fmt=DEFAULT_FMT): # clock(my_format) 傳進來的東西，在這裡被收進 self.fmt
        self.fmt = fmt

    def __call__(self, func):
        # __call__ 讓 clock 的實例「可被呼叫」；被呼叫時用 clocked 頂替原函式
        def clocked(*_args):
            # clocked 一樣是包住被裝飾函式的那層
            t0 = time.perf_counter()
            _result = func(*_args)
            elapsed = time.perf_counter() - t0
            name = func.__name__
            args = ', '.join(repr(arg) for arg in _args)
            result = repr(_result)
            print(self.fmt.format(**locals())) # 格式字串改從 self.fmt 取
            return _result
        return clocked

```

(這裡我刻意把類別名字寫成小寫的 clock，就是要強調它跟前面那個函式版是「可以直接替換、無痛上場」的。)

對照著看就很有意思。函式版是靠「函式包函式」疊出三層；類別版則把這三層的職責拆給了類別的兩個方法：`__init__` 扮演工廠那一層，負責把使用者傳進來的格式字串（`clock(my_format)`）收進 `self.fmt`，並回傳一個 clock 實例；而 `__call__` 扮演真正的裝飾器那一層，當你用 `@` 把這個實例套到函式上時，Python 就會去呼叫實例的 `__call__(func)`，讓它生出 `clocked` 來頂替原函式。原本要靠閉包抱住的格式字串，在類別版裡改由 `self` 這個實例屬性保管——殊途同歸，記住的东西一模一樣。

那到底該選哪種？沒有標準答案，看狀況：如果裝飾器要記的狀態很單純、邏輯也短，函式版輕巧直接、看起來最俐落；但如果裝飾器之後會長出更多狀態、或需要好幾個方法協同運作，類別版把東西收進 `self` 會更好讀、也更好擴充。兩種骨子裡是同一套「工廠生裝飾器」的思路，挑你看了最順眼、團隊最好維護的那種就對了。

到這裡，我們對「函式裝飾器」的整趟探索就告一段落了。至於「類別裝飾器」（用裝飾器去裝飾整個類別），那是後面另一堂課要講的主題，等走到那裡再會。

第四篇 用一級函式實作設計模式

在軟體工程裡，所謂的「設計模式（design pattern）」，就是一份用來解決某類常見設計難題的「通用食譜」。你完全不需要事先懂設計模式才能跟上這門課，課程裡會用到哪些模式，我都會一一講清楚。你可以把它想成武術的招式套路——前人把「碰到這種狀況該怎麼出招」整理成一套固定章法，後人遇到同樣的局面，照著套路打就行，不必每次都從零摸索。

設計模式這套東西會在程式設計界紅起來，要歸功於一本里程碑級的書：《Design Patterns: Elements of Reusable Object-Oriented Software》，作者是 Erich Gamma、Richard Helm、Ralph Johnson 跟 John Vlissides 四個人，江湖人稱「四人幫（the Gang of Four，簡稱 GoF）」。這本書本質上是一份目錄，收錄了 23 種模式，每一種都是某幾個類別的特定組合擺法，書裡是用 C++ 來示意的，但作者假定這些模式在其他物件導向語言裡同樣管用。

不過要注意：設計模式雖然號稱「跟語言無關」，但這不代表「每一種模式都適用於每一種語言」。舉個例子，後面某一堂課會講到——在 Python 裡硬要去模仿「迭代器（Iterator）模式」那套食譜，其實一點意義都沒有。為什麼？因為這個模式早就被內建進語言裡了，以「產生器（generator）」的形式現成擺在那兒等你用；產生器根本不必靠類別就能運作，寫起來也比那套經典食譜精簡太多。既然語言本身已經幫你把這道菜煮好端上桌了，你又何必再照著老食譜從頭炒一遍。

值得玩味的是，連《設計模式》的作者群自己，都在書的導論裡承認了一件事：你用的是哪種程式語言，會直接左右「哪些模式才真正重要」。他們大意是說，語言的選擇之所以關鍵，是因為它會形塑你看事情的角度；他們那套模式預設的是 Smalltalk、C++ 那個層級的語言特性，而這個前提就決定了「哪些東西容易實作、哪些不容易」。他們甚至坦言，要是當初假設的是程序式語言，搞不好就會多收幾個叫做「繼承」「封裝」「多型」的所謂模式進去——因為在那種語言裡，這些反而是要費力張羅的招式。同理，有些模式在某些比較冷門的物件導向語言裡是被「直接內建支援」的，比方說 CLOS 有「多重方法（multi-methods）」這種機制，這就大大降低了像「訪問者（Visitor）模式」那種招式的存在必要。換句話說，語言越強，有些模式就越顯得多此一舉。

順著這個脈絡，Peter Norvig 在他 1996 年那場著名的演講「Design Patterns in Dynamic Languages（動態語言裡的設計模式）」中拋出一個很有份量的觀察：在動態語言裡，原本《設計模式》那 23 種模式，有高達 16 種會變得「要嘛根本隱形、要嘛大幅簡化」。他講的雖然是 Lisp 跟 Dylan 這兩種語言，但他點名的那些動態特性，有不少在 Python 裡同樣具備。其中特別關鍵的是：在「支援一級函式（first-class

functions)」的語言脈絡下，Norvig 建議我們重新審視幾個經典模式——策略（Strategy）、命令（Command）、樣板方法（Template Method）、以及訪問者（Visitor）。

而這門課要帶你看清的，正是這件事：在某些情況下，函式其實能扛起跟類別一模一樣的工作，而且寫出來的程式碼更好讀、更精簡。我們接下來會親手把一份用類別實作的「策略模式」重構掉，改用「函式即物件」的思路來寫，順手砍掉一大堆樣板程式碼（boilerplate）；之後也會用類似的手法，聊聊怎麼把「命令模式」一併簡化下來。

個案研究：重構策略模式（Case Study: Refactoring Strategy）

策略（Strategy）模式正是一個好例子，告訴你「只要善用函式是一級物件這件事，在 Python 裡它可以變得多簡單」。接下來我會先用《設計模式》那本書描述的「經典」結構來實作策略模式；如果你本來就熟這套經典寫法，大可直接跳到後面「函式導向的策略」那一段，看我們怎麼用函式重構、把程式碼行數砍掉一大截。

先給你一個畫面。策略模式可以想成「結帳櫃台上有一個可以插不同折扣方案卡的插槽」：櫃台（Order）本身不曉得怎麼算折扣，它只負責把算折扣這件事「外包」出去，交給插進來的那張方案卡去處理；而那些方案卡（各種具體折扣）可以隨意抽換。《設計模式》對策略模式的定義，大意就是：定義一整組演算法，把每一個各自封裝起來、讓它們能互相替換，如此一來，演算法就能獨立於「使用它的那個客戶端」而自由變化。

拿電商場景來說，最清楚的策略應用就是「依顧客屬性或訂單內容來算折扣」。假設有一家網路商店，折扣規則如下：忠誠點數累積滿 1,000 點以上的顧客，整張訂單打 95 折；同一張訂單裡，任何一個品項只要數量達到 20 件以上，該品項就打 9 折；而只要一張訂單包含 10 種以上不同的品項，整單打 93 折。為了簡化，我們先假設一張訂單最多只套用一種折扣。

在那張 UML 類別圖裡，有三個角色。「情境（Context）」負責對外提供服務，但把某段運算委派給「可替換、各自實作不同演算法」的元件——在這個電商情境裡，情境就是 Order，它會被設定成依其中某一種演算法來套用促銷折扣。「策略（Strategy）」是那些元件共通的介面，在這裡由一個叫 Promotion 的抽象類別擔綱。「具體策略（Concrete strategy）」則是策略底下一個個具體的子類別，也就是我們要實作的 FidelityPromo、BulkItemPromo、LargeOrderPromo 這三張方案卡。

要特別講一下：具體策略選哪一張，是由「情境類別的客戶端」決定的。在我們的情況裡，系統會在建立訂單之前，先以某種方式選好一個促銷折扣策略，再把它傳給 Order 的建構式。至於「怎麼選策略」這件事本身，則落在這個模式的管轄範圍之外。經典版的程式碼如下：

```
from abc import ABC, abstractmethod
from collections.abc import Sequence
```

```

from decimal import Decimal
from typing import NamedTuple, Optional

class Customer(NamedTuple):
    name: str
    fidelity: int

class LineItem(NamedTuple):
    product: str
    quantity: int
    price: Decimal

    def total(self) -> Decimal:
        return self.price * self.quantity

class Order(NamedTuple):
    customer: Customer
    cart: Sequence[LineItem]
    promotion: Optional['Promotion'] = None

    # 這就是「情境(Context)」

    def total(self) -> Decimal:
        totals = (item.total() for item in self.cart)
        return sum(totals, start=Decimal(0))

    def due(self) -> Decimal:
        if self.promotion is None:
            discount = Decimal(0)
        else:
            discount = self.promotion.discount(self) # 把算折扣外包給插進來的方案卡
        return self.total() - discount

    def __repr__(self):
        return f'<Order total: {self.total():.2f} due: {self.due():.2f}>'

```

```

class Promotion(ABC):                                # 這是「策略」：一個抽象基底類別，定義共通介面
    @abstractmethod
    def discount(self, order: Order) -> Decimal:
        """以正數的金額形式回傳折扣"""

class FidelityPromo(Promotion):                      # 第一張具體策略卡
    """忠誠點數滿 1000 點以上的顧客，整單打 95 折"""
    def discount(self, order: Order) -> Decimal:
        rate = Decimal('0.05')
        if order.customer.fidelity >= 1000:
            return order.total() * rate
        return Decimal(0)

class BulkItemPromo(Promotion):                     # 第二張具體策略卡
    """單一品項數量達 20 件以上者，該品項打 9 折"""
    def discount(self, order: Order) -> Decimal:
        discount = Decimal(0)
        for item in order.cart:
            if item.quantity >= 20:
                discount += item.total() * Decimal('0.1')
        return discount

class LargeOrderPromo(Promotion):                   # 第三張具體策略卡
    """訂單含 10 種以上不同品項，整單打 93 折"""
    def discount(self, order: Order) -> Decimal:
        distinct_items = {item.product for item in order.cart}
        if len(distinct_items) >= 10:
            return order.total() * Decimal('0.07')
        return Decimal(0)

```

這裡我刻意把 Promotion 寫成抽象基底類別 (ABC)，好用上 `@abstractmethod` 這個裝飾器，讓「這是個介面、子類別非實作 discount 不可」這件事更白紙黑字。

接著用 doctest 來示範並驗證這套規則跑起來對不對：

```
>>> joe = Customer('John Doe', 0) # joe 有 0 點忠誠點數
>>> ann = Customer('Ann Smith', 1100) # ann 有 1,100 點
>>> cart = (LineItem('banana', 4, Decimal('.5')),
...         LineItem('apple', 10, Decimal('1.5')),
...         LineItem('watermelon', 5, Decimal(5)))
>>> Order(joe, cart, FidelityPromo()) # joe 點數不夠，忠誠折扣給他 0 折
<Order total: 42.00 due: 42.00>
>>> Order(ann, cart, FidelityPromo()) # ann 點數夠，整單享 5% 折扣
<Order total: 42.00 due: 39.90>
>>> banana_cart = (LineItem('banana', 30, Decimal('.5')),
...                LineItem('apple', 10, Decimal('1.5')))
>>> Order(joe, banana_cart, BulkItemPromo()) # 香蕉 30 件，靠大量折扣省了 1.5 元
<Order total: 30.00 due: 28.50>
>>> long_cart = tuple(LineItem(str(sku), 1, Decimal(1))
...                   for sku in range(10))
>>> Order(joe, long_cart, LargeOrderPromo()) # 10 種不同品項，整單 7% 折扣
<Order total: 10.00 due: 9.30>
>>> Order(joe, cart, LargeOrderPromo()) # 品項種類不到 10 種，沒折扣
<Order total: 42.00 due: 42.00>
```

這套經典寫法跑得好好的，沒問題。但同樣的功能，在 Python 裡用「函式即物件」的思路，可以用更少的程式碼做到。

函式導向的策略（Function-Oriented Strategy）

仔細看上面那三張具體策略卡：每一張都是一個「只有一個方法（discount）」的類別，而且這些策略實例「完全沒有狀態」（沒有任何實例屬性）。你會忍不住覺得它們長得超像「就只是一個普通函式」——而你想得完全沒錯。

下面就是重構後的版本：把那三個具體策略類別直接換成簡單的函式，連 Promotion 抽象類別都整個拿掉；Order 類別只需要動一點點小手術：

```

from collections.abc import Sequence
from dataclasses import dataclass
from decimal import Decimal
from typing import Optional, Callable, NamedTuple

class Customer(NamedTuple):
    name: str
    fidelity: int

class LineItem(NamedTuple):
    product: str
    quantity: int
    price: Decimal

    def total(self):
        return self.price * self.quantity

@dataclass(frozen=True)
class Order:                                # 一樣是「情境」
    customer: Customer
    cart: Sequence[LineItem]
    promotion: Optional[Callable[['Order'], Decimal]] = None  # 型別註記改了：見下方說明

    def total(self) -> Decimal:
        totals = (item.total() for item in self.cart)
        return sum(totals, start=Decimal(0))

    def due(self) -> Decimal:
        if self.promotion is None:
            discount = Decimal(0)
        else:
            discount = self.promotion(self)        # 直接把 self.promotion 當函式呼叫，傳入 self
        return self.total() - discount

```

```

def __repr__(self):
    return f'<Order total: {self.total():.2f} due: {self.due():.2f}>'

def fidelity_promo(order: Order) -> Decimal:      # 每個策略就是一個函式
    """忠誠點數滿 1000 點以上的顧客，整單打 95 折"""
    if order.customer.fidelity >= 1000:
        return order.total() * Decimal('0.05')
    return Decimal(0)

def bulk_item_promo(order: Order) -> Decimal:
    """單一品項數量達 20 件以上者，該品項打 9 折"""
    discount = Decimal(0)
    for item in order.cart:
        if item.quantity >= 20:
            discount += item.total() * Decimal('0.1')
    return discount

def large_order_promo(order: Order) -> Decimal:
    """訂單含 10 種以上不同品項，整單打 93 折"""
    distinct_items = {item.product for item in order.cart}
    if len(distinct_items) >= 10:
        return order.total() * Decimal('0.07')
    return Decimal(0)

```

兩處改動值得點名。第一，promotion 的型別註記變成了 `Optional[Callable[['Order'], Decimal]]`，白話就是：promotion 要嘛是 None，要嘛是「一個吃 Order、吐回 Decimal 的可呼叫物件」。第二，算折扣那行從原本的 `self.promotion.discount(self)` 簡化成 `self.promotion(self)`——因為現在 promotion 本身就是個函式，直接呼叫它、把 self 傳進去就好。整段沒有抽象類別，每個策略就是赤裸裸的一個函式。

用起來也跟著變清爽。最大的差別在於：你傳的是「函式本身」，而不是「函式呼叫的結果」，所以是 `fidelity_promo` 而不是 `FidelityPromo()`——後面那對括號不見了：

```

>>> joe = Customer('John Doe', 0)
>>> ann = Customer('Ann Smith', 1100)
>>> cart = [LineItem('banana', 4, Decimal('.5')),
...         LineItem('apple', 10, Decimal('1.5')),
...         LineItem('watermelon', 5, Decimal(5))]
>>> Order(joe, cart, fidelity_promo)           # 直接把函式當引數傳，不必建立物件
<Order total: 42.00 due: 42.00>
>>> Order(ann, cart, fidelity_promo)
<Order total: 42.00 due: 39.90>
>>> banana_cart = [LineItem('banana', 30, Decimal('.5')),
...                 LineItem('apple', 10, Decimal('1.5'))]
>>> Order(joe, banana_cart, bulk_item_promo)
<Order total: 30.00 due: 28.50>
>>> long_cart = [LineItem(str(item_code), 1, Decimal(1))
...              for item_code in range(10)]
>>> Order(joe, long_cart, large_order_promo)
<Order total: 10.00 due: 9.30>
>>> Order(joe, cart, large_order_promo)
<Order total: 42.00 due: 42.00>

```

注意這裡的關鍵：每來一張新訂單，你「不需要再 new 一個新的促銷物件」出來——這些函式本身就是現成的，拿了就能用。

這裡有個很有意思的轉折，正好戳中經典模式的痛點。《設計模式》的作者群其實有建議：策略物件往往很適合做成「享元（flyweight）」；而他們對享元的描述大意是「一個能被多個情境同時共用的共享物件」。他們之所以建議共用，是因為當「同一個策略要被反覆套用到一個個新情境（在我們這就是一張張新訂單）」時，每次都重新建立具體策略物件是有成本的。換句話說，為了補救策略模式的一個缺點（執行期建物件的成本），作者居然得再疊上「另一個模式」來救——而你的程式碼行數跟維護成本，就在這一層層疊加中越堆越高。

但反過來想就豁然開朗了：如果你的具體策略「根本沒有內部狀態」、它只處理來自情境的資料，那就大大方方用樸素的函式吧，別再傻傻地去寫一個「只有單一方法的類別、去實作另一個類別所宣告的單一方法介面」。函式比自訂類別的實例輕量太多，而且這裡壓根不需要享元——因為每個策略函式在 Python 行程載入模組時就「只被建立一次」，它本身，就已經是那個「能被多個情境同時共用的共享物件」了。一個函式，一槍解決了人家兩個模式才搞定的事。

把策略模式用函式實作出來之後，新的可能性也跟著冒出來。假設你想做一個「元策略（metastategy）」——它能自動替某張訂單挑出「所有折扣裡最划算的那個」，該怎麼辦？接下來幾段，我們就用各種「把函式與模組都當成物件」的手法，一步步重構出這個需求。

挑出最佳策略：簡單做法（Choosing the Best Strategy: Simple Approach）

沿用前面那幾個顧客跟購物車，我們補三個測試，看 `best_promo` 這個「自動挑最划算」的東西表現如何：

```
>>> Order(joe, long_cart, best_promo)           # 替 joe 自動挑中了大量品項折扣
<Order total: 10.00 due: 9.30>
>>> Order(joe, banana_cart, best_promo)         # 香蕉買很多，自動挑中大量商品折扣
<Order total: 30.00 due: 28.50>
>>> Order(ann, cart, best_promo)                 # 簡單購物車，忠誠的 ann 自動拿到忠誠折扣
<Order total: 42.00 due: 39.90>
```

而 `best_promo` 的實作簡單到不可思議：

```
promos = [fidelity_promo, bulk_item_promo, large_order_promo]  # 一個裝著「函式」的串列

def best_promo(order: Order) -> Decimal:
    """算出這張訂單可享的最佳折扣"""
    return max(promo(order) for promo in promos)  # 把每個策略都套一遍，取折扣最大的那個
```

它直白得很：`promos` 就是一個「裝著函式的串列」，`best_promo` 用一個產生器運算式把每個函式都套到訂單上，再回傳算出來最大的那個折扣。一旦你習慣「函式是一級物件」這個觀念，「建立一個資料結構來裝函式」這種事就會變得再自然不過。

不過這版雖然能跑又好讀，卻藏著一個會釀成隱性 bug 的重複：每次要新增一種促銷策略，你不但得寫那個函式，還得「記得」把它手動加進 `promos` 串列。萬一你忘了加，這個新折扣在「被明確當引數傳給 `Order`」時還是能用，卻會被 `best_promo` 完全無視——因為它根本不在那張清單上。這種「人工同步兩個地方」的設計遲早出包。下面就來看兩種解法。

在模組裡自動找出策略（Finding Strategies in a Module）

在 Python 裡，模組本身也是一級物件，標準函式庫還附了好幾個處理模組的函式。其中內建的 `globals` 大意是這樣：它會回傳一個字典，代表「當下的全域符號表」——而這永遠是「當前這個模組」的字典（在函式或方法內部呼叫時，指的是它被定義的那個模組，而不是呼叫它的那個模組）。

利用這點，我們可以耍個小花招，讓 `best_promo` 自己去「翻找」同模組裡所有可用的 `*_promo` 函式：

```
from decimal import Decimal
from strategy import Order
from strategy import (
    fidelity_promo, bulk_item_promo, large_order_promo,
)

promos = [promo for name, promo in globals().items()
          if name.endswith('_promo') and
          name != 'best_promo']

def best_promo(order: Order) -> Decimal:
    """算出這張訂單可享的最佳折扣"""
    return max(promo(order) for promo in promos)
```

遍歷 globals() 字典裡的每一項
只挑名字以 _promo 結尾的
但要排除 best_promo 自己，否則無限遞迴

best_promo 本身完全沒變

這裡先把那幾個促銷函式匯入進來，讓它們出現在全域命名空間裡；接著掃過 `globals()` 回傳的字典，只留下「名字以 `_promo` 結尾」的值，並且特地把 `best_promo` 自己過濾掉——不然 `best_promo` 呼叫時又把自己算進去，就會掉進無限遞迴的坑。

另一種收集促銷函式的做法，是乾脆開一個獨立模組，把所有策略函式（就 `best_promo` 例外）全擺進去。這版唯一的重點改動，是改成「內省 (introspection) 一個叫 `promotions` 的獨立模組」來建出策略清單。它得多匯入 `promotions` 模組，以及提供高階內省功能的 `inspect`：

```
from decimal import Decimal
import inspect
from strategy import Order
import promotions
```

```
promos = [func for _, func in inspect.getmembers(promotions, inspect.isfunction)]

def best_promo(order: Order) -> Decimal:
    """算出這張訂單可享的最佳折扣"""
    return max(promo(order) for promo in promos)
```

這裡的 `inspect.getmembers` 會回傳某個物件（這裡就是 `promotions` 模組）的所有屬性，並且可以選擇用一個述詞（predicate，也就是一個回傳布林值的函式）來過濾；我們用 `inspect.isfunction` 把模組裡的「函式」全撈出來。

這版的好處是：不管那些函式叫什麼名字都無所謂，唯一的要求就是 `promotions` 模組裡「只能放算折扣的函式」。當然，這是程式碼一個沒明說的隱性假設——萬一有人在 `promotions` 模組裡塞了一個簽名不一樣的函式，`best_promo` 一旦想把它套到訂單上就會炸掉。你當然可以加更嚴格的檢查，比方說連函式的引數都拿來內省一番再決定收不收。不過這版的重點不在於給你一個滴水不漏的完整解答，而是要凸顯「模組內省」這條路的一種可能玩法。

至於比「動態自動收集」更明確、更直白的另一條路，是用一個簡單的裝飾器來主動登記促銷函式——那正是下一段要講的東西。

用裝飾器強化的策略模式：讓促銷方案自己「報到」

先講個情境你就懂了。

想像你是某個學生社團的社長，社團每學期會推出好幾個優惠活動。以前的做法很土法煉鋼：每個活動方案你都要先寫好，然後再回頭手動抄一份「活動清單」，之後挑「哪個優惠最划算」的時候就看那張清單。問題就出在這——只要哪天有學弟妹新增了一個活動，卻忘記回去把它抄進清單，那這個活動就等於不存在，挑優惠的時候永遠輪不到它。最慘的是程式不會報錯，它只是默默忽略，變成那種很難抓的暗坑。

新做法聰明多了：在報到桌擺一張「報到表」，每個活動方案寫完之後，自己走過報到桌蓋個章，就自動被登記進清單。社長完全不用記、不用抄，只要看那張表就好，永遠不會漏。`@promotion` 這個裝飾器就是那張報到桌，被它蓋過章的函式，通通自動進清單。

這正是前面課程那版做法的痛點所在：函式名稱要重複寫兩次（一次定義、一次塞進 `promos` 清單），只要有人加了新策略卻忘了手動登記，`best_promo` 就會悄悄略過它。這版用「註冊裝飾器」的技巧，把這個漏洞補起來。

優化後的程式（逐行註解）

```

from collections.abc import Callable # 拿來描述「可被呼叫的東西」的型別，等下宣告函式長相要用
from decimal import Decimal         # 金額一律用 Decimal，避開浮點數誤差(不然 0.1+0.2 會跑出怪數字)

Promotion = Callable[[Order], Decimal] # 幫「吃一個 Order、吐回一個 Decimal」的函式取個別名，後面好讀

promos: list[Promotion] = []          # 模組層級的全域清單，一開始空的，等著被報到登記

def promotion(promo: Promotion) -> Promotion: # 這就是「報到桌」裝飾器，收進一個促銷函式
    promos.append(promo)                     # 關鍵就這行：把函式登記進清單
    return promo                             # 原封不動還回去，完全不改它的行為

def best_promo(order: Order) -> Decimal:      # 負責挑出最划算折扣的人
    """算出這筆訂單能拿到的最高折扣"""
    return max(promo(order) for promo in promos) # 清單裡每個方案都套一遍，取最大值

@promotion                                  # 蓋章報到，fidelity 自動進清單
def fidelity(order: Order) -> Decimal:
    """忠誠點數滿 1000 給 5% 折扣"""
    if order.customer.fidelity >= 1000:
        return order.total() * Decimal('0.05')
    return Decimal(0)                       # 不符合就回 0，代表這方案這次沒折扣

@promotion
def bulk_item(order: Order) -> Decimal:
    """同一品項買滿 20 件，那一項折 10%"""
    discount = Decimal(0)
    for item in order.cart:                 # 逐項翻購物車
        if item.quantity >= 20:
            discount += item.total() * Decimal('0.1') # 符合的品項折扣累加起來
    return discount

@promotion
def large_order(order: Order) -> Decimal:

```

```
"""整單有 10 種以上不同商品，全單折 7%"""
distinct_items = {item.product for item in order.cart} # 用集合去重，算出到底有幾「種」商品
if len(distinct_items) >= 10:
    return order.total() * Decimal('0.07')
return Decimal(0)
```

幾個比較容易卡住的地方

`promotion` 這個裝飾器最妙的是它「什麼都沒改」。它收到函式、塞進清單、然後把函式整顆還回去。所以 `fidelity`、`bulk_item`、`large_order` 被裝飾之後，你照樣可以單獨呼叫它們，行為一模一樣，差別只在於它們「順手」被登記了。這種不改變被裝飾物、只做登記動作的，就叫註冊裝飾器。

`best_promo` 這支從頭到尾完全不用動，因為它只認 `promos` 這張清單，不在乎裡面的方案是怎麼進來的。這就是這個寫法的厲害之處：新增方案跟挑方案兩件事被切得乾乾淨淨。

`large_order` 裡那行 `{item.product for item in order.cart}` 是集合推導式，重點在「去重」——同一種商品買兩次也只算一種，所以才能用 `len()` 數出「幾種不同商品」。

報到流程示意 (PlantUML)



這個寫法贏在哪

促銷函式的名字可以隨便取，不必再硬掛 `_promo` 這種後綴去配合清單；而且這些策略函式可以散在系統任何角落、寫在別的模組裡，只要記得蓋上 `@promotion` 的章，它就會自己報到。社長（`best_promo`）永遠看得到最新的全部方案。

更高階的做法：把清單收進物件裡

上面那版有個小遺憾——`promos` 是裸露在模組層級的全域變數，任何地方都能亂動它。同樣的邏輯、同樣的條件，可以再升一階，用一個登記器物件把清單包起來，讓狀態不要散在外面：

```
class PromoRegistry:
    def __init__(self) -> None:
        self._promos: list[Promotion] = []           # 清單藏進物件裡，不再裸奔在模組層級
```

```

def register(self, promo: Promotion) -> Promotion: # 一樣當裝飾器用
    self._promos.append(promo)                    # 登記動作不變
    return promo                                  # 一樣原封不動還回去

def best(self, order: Order) -> Decimal:
    """算出最高折扣，邏輯跟原本完全一致"""
    return max(promo(order) for promo in self._promos)

registry = PromoRegistry()                        # 開一個登記器實體

@registry.register                               # 改用物件的方法當報到桌
def fidelity(order: Order) -> Decimal:
    """忠誠點數滿 1000 給 5% 折扣"""
    if order.customer.fidelity >= 1000:
        return order.total() * Decimal('0.05')
    return Decimal(0)

```

折扣條件、最高折扣的算法，跟前面那版一字不差，只是把「那張清單」跟「報到、挑選」這些行為全部收進 `PromoRegistry` 裡。好處是你能同時開好幾個獨立的登記器（像不同分店各有各的優惠池），彼此不會打架，而且清單不會被外面誤動。

接下來的課程會接著聊另一個設計模式：命令模式（Command）——它有時候會被人寫成只有一個方法的類別，但其實很多時候用單純的函式就夠了。